

# *Algorithmique 1*

## *4. Algorithmes de tri – suite*

Johan Thapper  
2023–2024



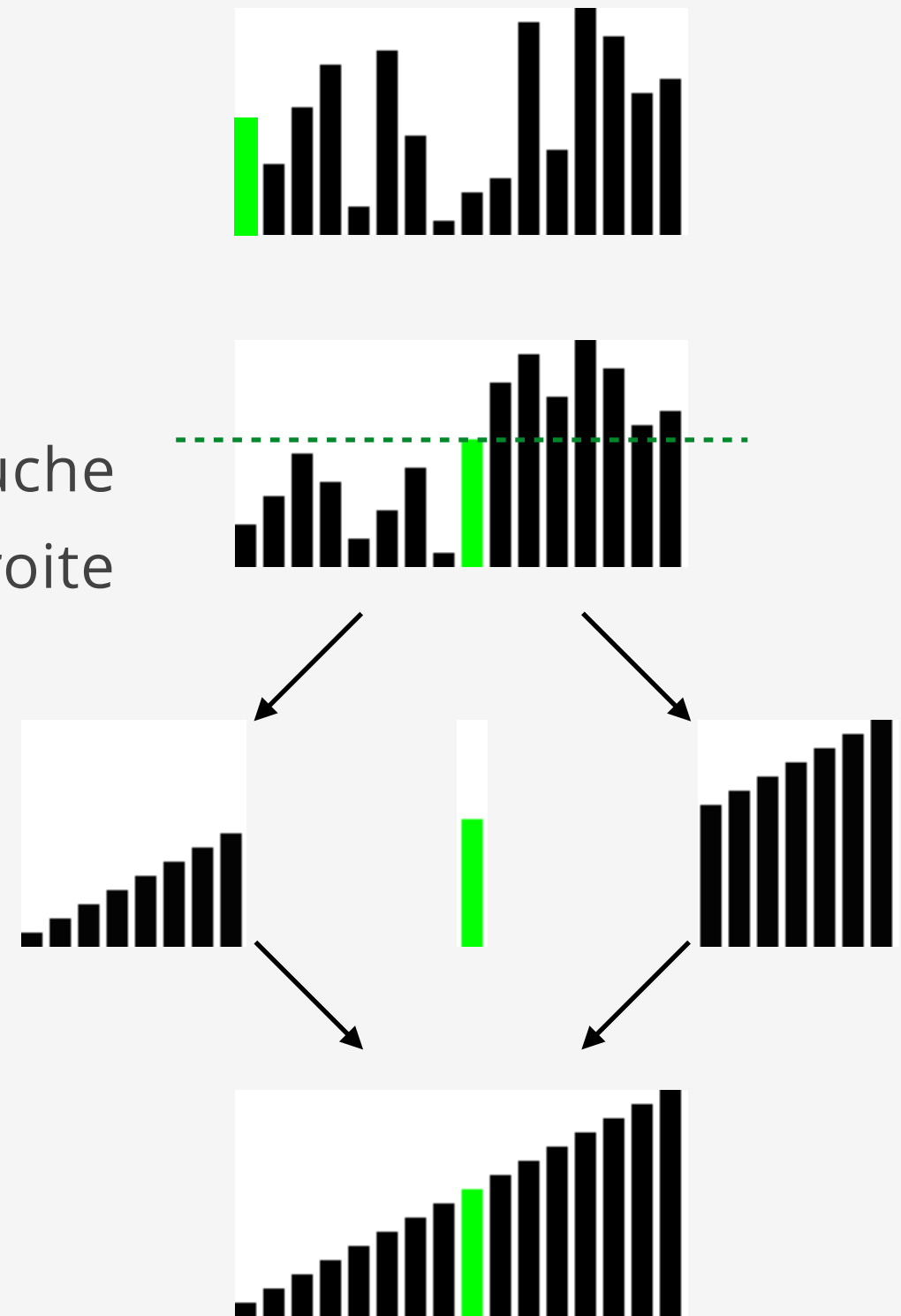
# Comparaison

|              |              | tri par insertion | tri par sélection |
|--------------|--------------|-------------------|-------------------|
| pire cas     | comparaisons | $n(n-1)/2$        | $n(n-1)/2$        |
|              | échanges     | $n(n-1)/2$        | $n-1$             |
|              | complexité   | $O(n^2)$          | $O(n^2)$          |
| meilleur cas | comparaisons | $n-1$             | $n(n-1)/2$        |
|              | échanges     | 0                 | 0                 |
|              | complexité   | $O(n)$            | $O(n^2)$          |

- ▶ Tri par insertion
  - ❖ efficace pour des petits tableaux et des tableaux partiellement triés
- ▶ Tri par sélection
  - ❖ efficace si les échanges coûtent cher

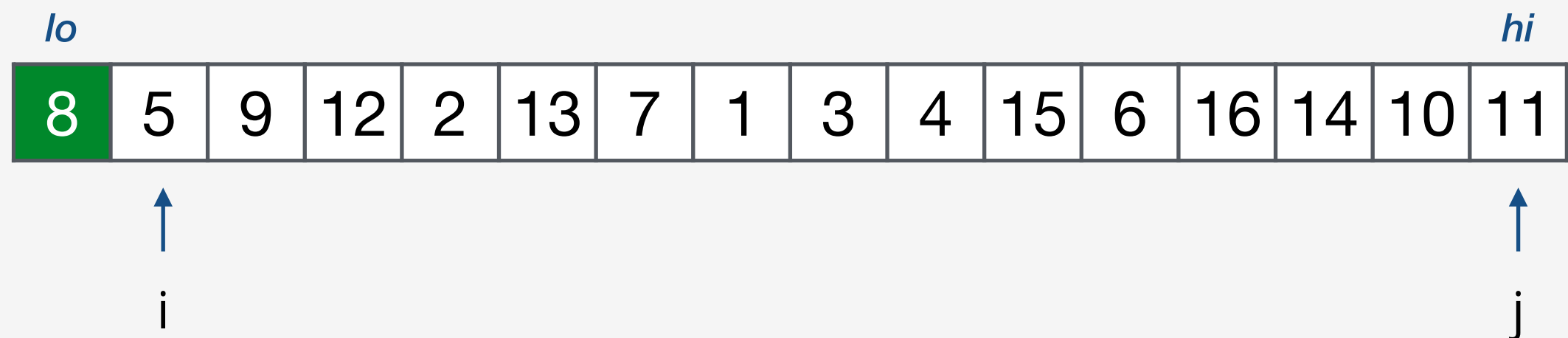
# Quicksort

- ▶ sélectionner **un pivot**
- ▶ partitionner le tableau pour que
  - ❖ **le pivot** soit à sa position finale  $t[j]$
  - ❖ les éléments plus petits soient à gauche
  - ❖ les éléments plus grands soient à droite
- ▶ trier récursivement
  - ❖  $t[lo], \dots, t[j-1]$  et
  - ❖  $t[j+1], \dots, t[hi]$



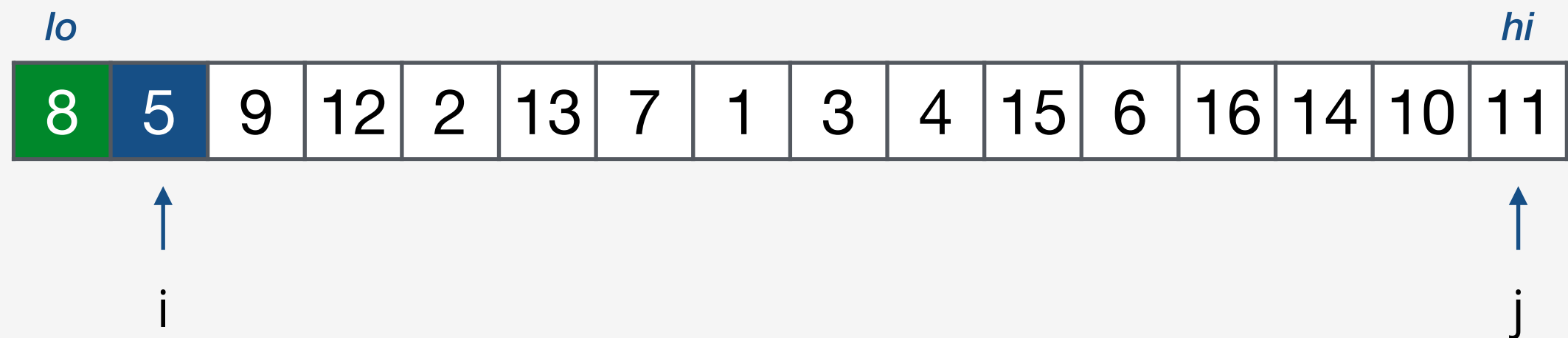
# Partitionnement

- ▶ Répéter jusqu'à ce que  $i$  et  $j$  se croisent
  - ❖ incrémenter l'indice  $i$  tant que  $t[i] < t[lo]$
  - ❖ décrémenter l'indice  $j$  tant que  $t[j] > t[lo]$
  - ❖ échanger  $t[i]$  et  $t[j]$ , incrémenter  $i$ , décrémenter  $j$
- ▶ Quand  $i$  et  $j$  se croisent
  - ❖ échanger  $t[lo]$  et  $t[j]$



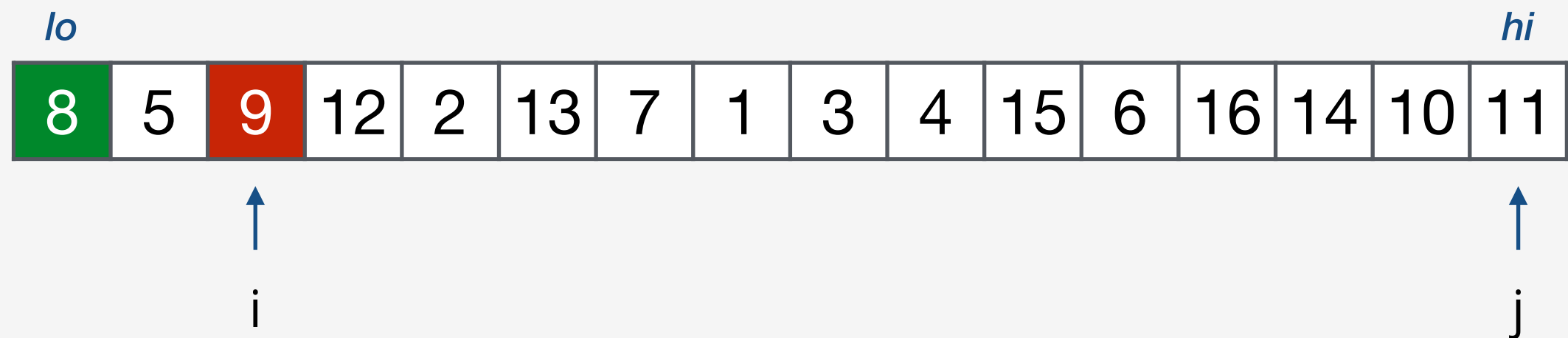
# Partitionnement

- ▶ Répéter jusqu'à ce que  $i$  et  $j$  se croisent
  - ❖ incrémenter l'indice  $i$  tant que  $t[i] < t[lo]$
  - ❖ décrémenter l'indice  $j$  tant que  $t[j] > t[lo]$
  - ❖ échanger  $t[i]$  et  $t[j]$ , incrémenter  $i$ , décrémenter  $j$
- ▶ Quand  $i$  et  $j$  se croisent
  - ❖ échanger  $t[lo]$  et  $t[j]$



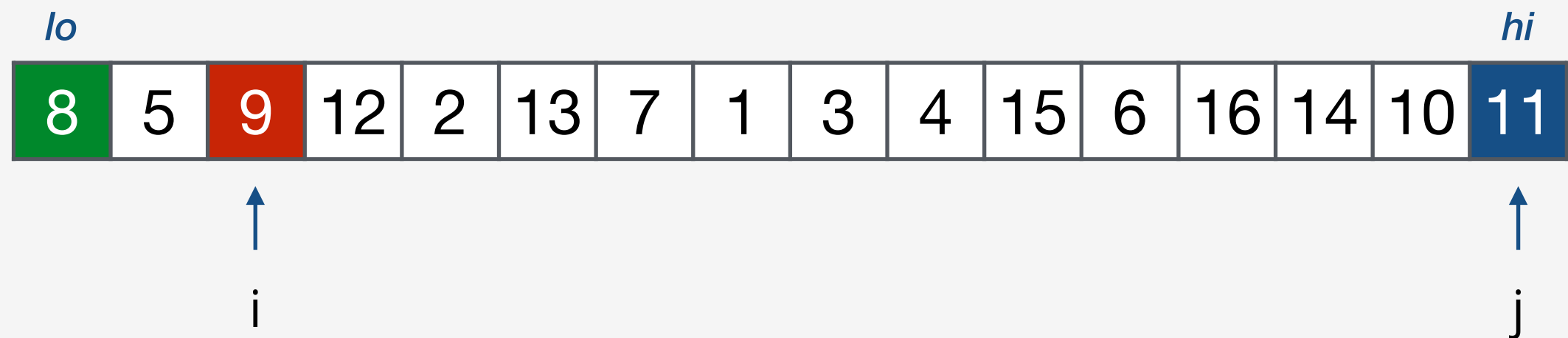
# Partitionnement

- ▶ Répéter jusqu'à ce que  $i$  et  $j$  se croisent
  - ❖ incrémenter l'indice  $i$  tant que  $t[i] < t[lo]$
  - ❖ décrémenter l'indice  $j$  tant que  $t[j] > t[lo]$
  - ❖ échanger  $t[i]$  et  $t[j]$ , incrémenter  $i$ , décrémenter  $j$
- ▶ Quand  $i$  et  $j$  se croisent
  - ❖ échanger  $t[lo]$  et  $t[j]$



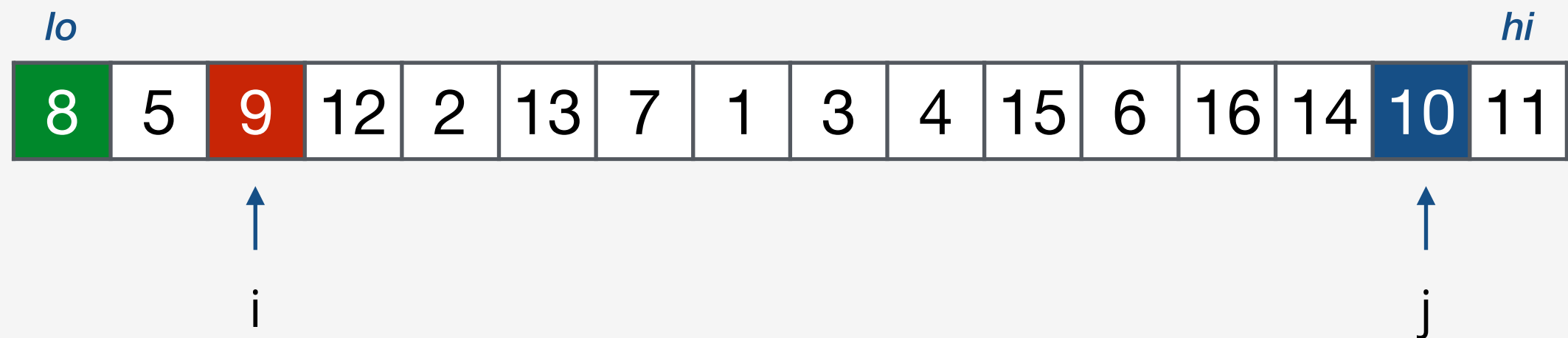
# Partitionnement

- ▶ Répéter jusqu'à ce que  $i$  et  $j$  se croisent
  - ❖ incrémenter l'indice  $i$  tant que  $t[i] < t[lo]$
  - ❖ décrémenter l'indice  $j$  tant que  $t[j] > t[lo]$
  - ❖ échanger  $t[i]$  et  $t[j]$ , incrémenter  $i$ , décrémenter  $j$
- ▶ Quand  $i$  et  $j$  se croisent
  - ❖ échanger  $t[lo]$  et  $t[j]$



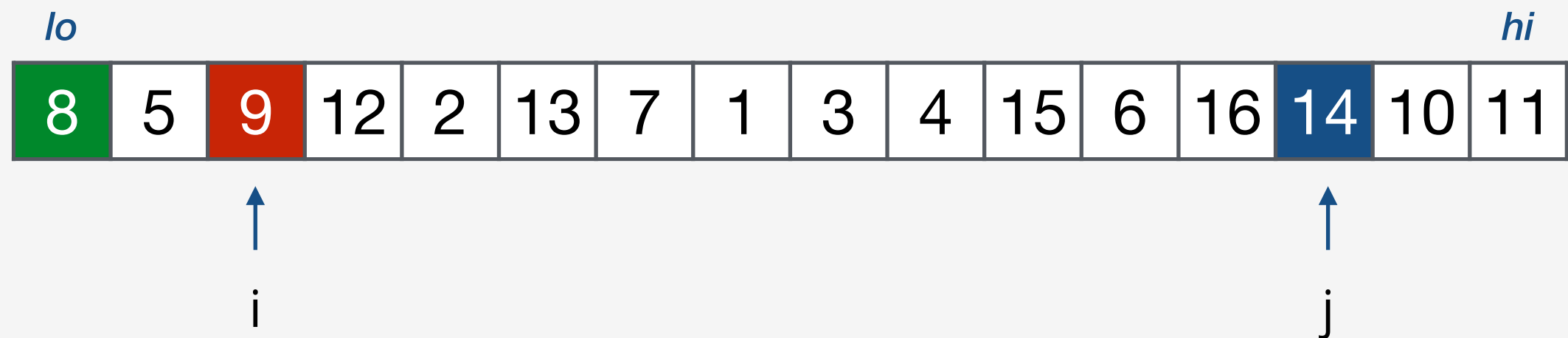
# Partitionnement

- ▶ Répéter jusqu'à ce que  $i$  et  $j$  se croisent
  - ❖ incrémenter l'indice  $i$  tant que  $t[i] < t[lo]$
  - ❖ décrémenter l'indice  $j$  tant que  $t[j] > t[lo]$
  - ❖ échanger  $t[i]$  et  $t[j]$ , incrémenter  $i$ , décrémenter  $j$
- ▶ Quand  $i$  et  $j$  se croisent
  - ❖ échanger  $t[lo]$  et  $t[j]$



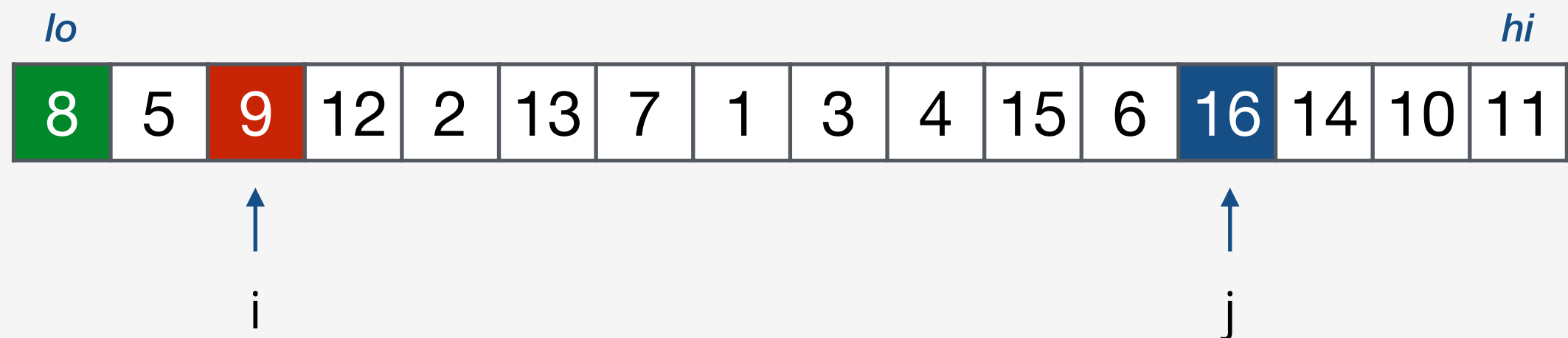
# Partitionnement

- ▶ Répéter jusqu'à ce que  $i$  et  $j$  se croisent
  - ❖ incrémenter l'indice  $i$  tant que  $t[i] < t[lo]$
  - ❖ décrémenter l'indice  $j$  tant que  $t[j] > t[lo]$
  - ❖ échanger  $t[i]$  et  $t[j]$ , incrémenter  $i$ , décrémenter  $j$
- ▶ Quand  $i$  et  $j$  se croisent
  - ❖ échanger  $t[lo]$  et  $t[j]$



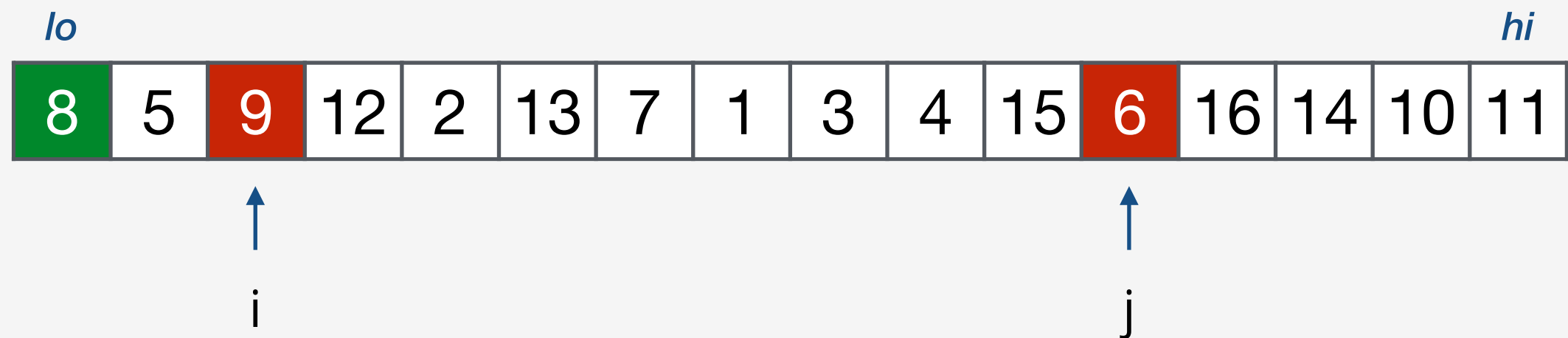
# Partitionnement

- ▶ Répéter jusqu'à ce que  $i$  et  $j$  se croisent
  - ❖ incrémenter l'indice  $i$  tant que  $t[i] < t[lo]$
  - ❖ décrémenter l'indice  $j$  tant que  $t[j] > t[lo]$
  - ❖ échanger  $t[i]$  et  $t[j]$ , incrémenter  $i$ , décrémenter  $j$
- ▶ Quand  $i$  et  $j$  se croisent
  - ❖ échanger  $t[lo]$  et  $t[j]$



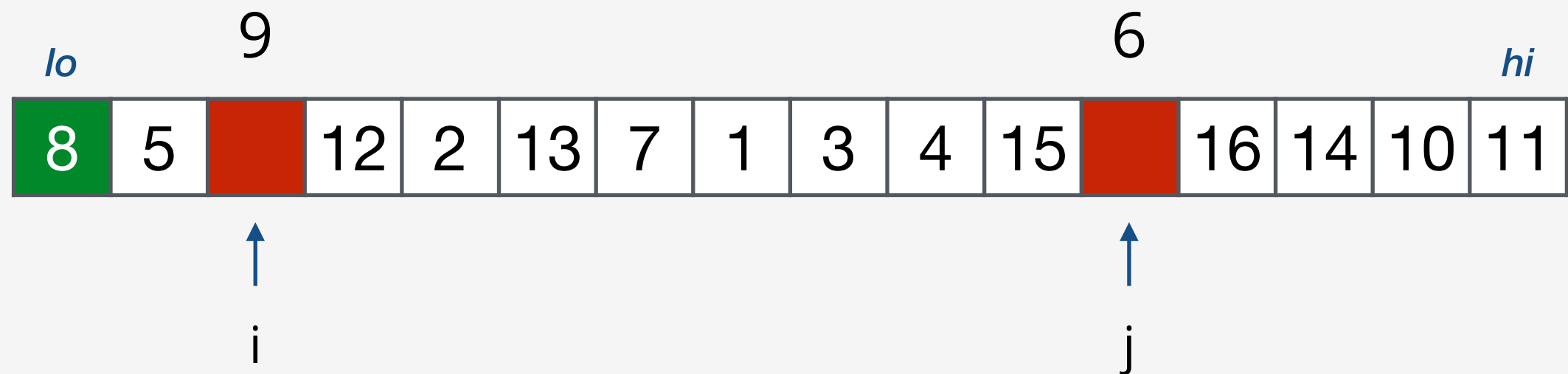
# Partitionnement

- ▶ Répéter jusqu'à ce que  $i$  et  $j$  se croisent
  - ❖ incrémenter l'indice  $i$  tant que  $t[i] < t[lo]$
  - ❖ décrémenter l'indice  $j$  tant que  $t[j] > t[lo]$
  - ❖ échanger  $t[i]$  et  $t[j]$ , incrémenter  $i$ , décrémenter  $j$
- ▶ Quand  $i$  et  $j$  se croisent
  - ❖ échanger  $t[lo]$  et  $t[j]$



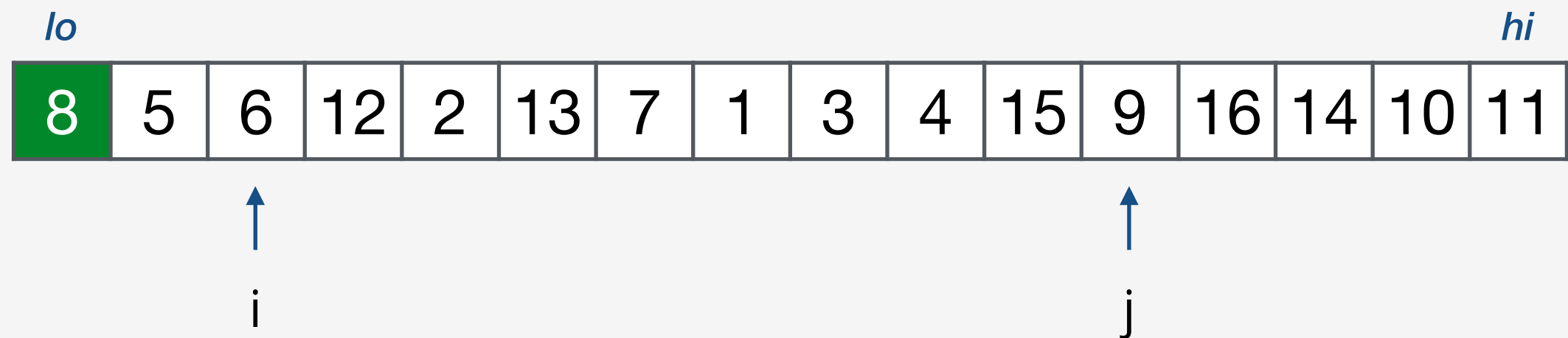
# Partitionnement

- ▶ Répéter jusqu'à ce que  $i$  et  $j$  se croisent
  - ❖ incrémenter l'indice  $i$  tant que  $t[i] < t[lo]$
  - ❖ décrémenter l'indice  $j$  tant que  $t[j] > t[lo]$
  - ❖ échanger  $t[i]$  et  $t[j]$ , incrémenter  $i$ , décrémenter  $j$
- ▶ Quand  $i$  et  $j$  se croisent
  - ❖ échanger  $t[lo]$  et  $t[j]$



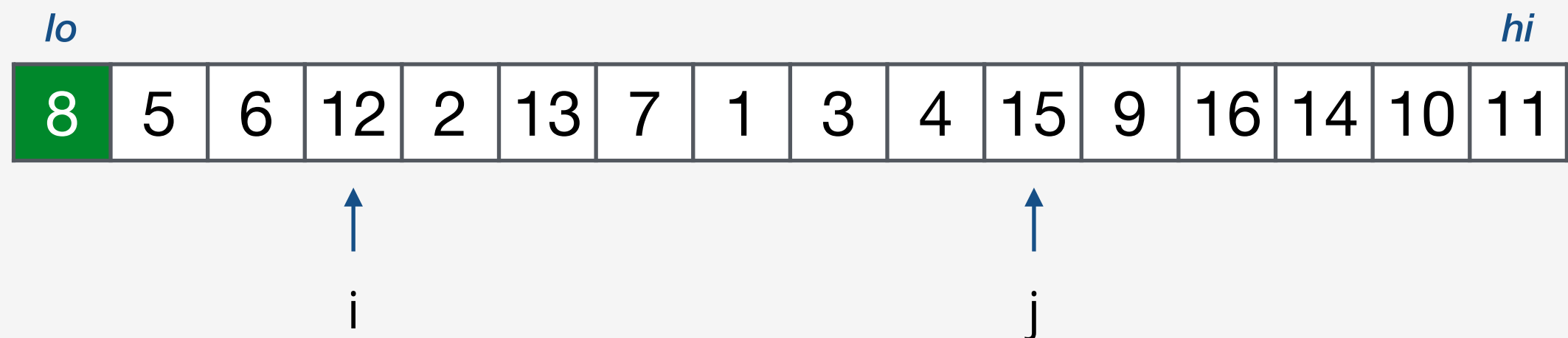
# Partitionnement

- ▶ Répéter jusqu'à ce que  $i$  et  $j$  se croisent
  - ❖ incrémenter l'indice  $i$  tant que  $t[i] < t[lo]$
  - ❖ décrémenter l'indice  $j$  tant que  $t[j] > t[lo]$
  - ❖ échanger  $t[i]$  et  $t[j]$ , incrémenter  $i$ , décrémenter  $j$
- ▶ Quand  $i$  et  $j$  se croisent
  - ❖ échanger  $t[lo]$  et  $t[j]$



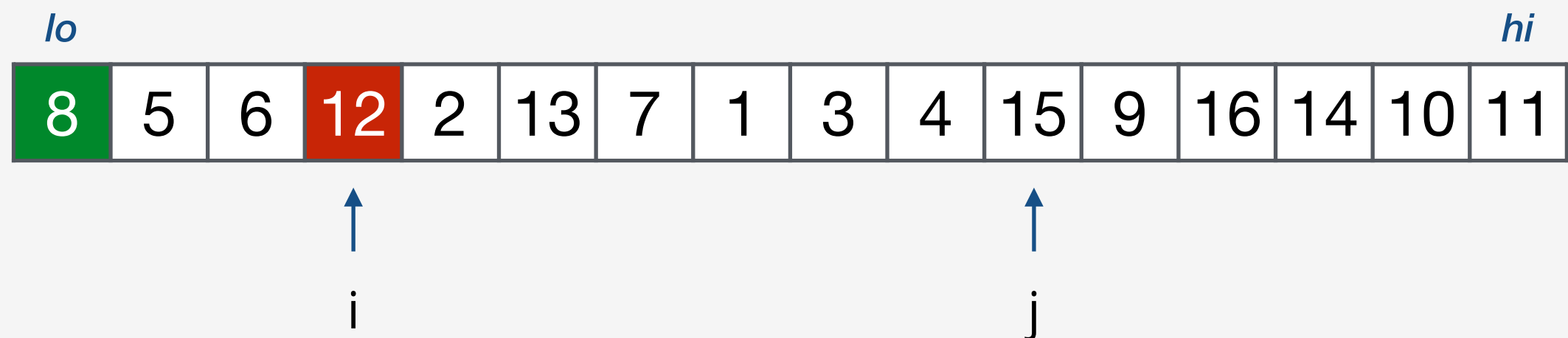
# Partitionnement

- ▶ Répéter jusqu'à ce que  $i$  et  $j$  se croisent
  - ❖ incrémenter l'indice  $i$  tant que  $t[i] < t[lo]$
  - ❖ décrémenter l'indice  $j$  tant que  $t[j] > t[lo]$
  - ❖ échanger  $t[i]$  et  $t[j]$ , incrémenter  $i$ , décrémenter  $j$
- ▶ Quand  $i$  et  $j$  se croisent
  - ❖ échanger  $t[lo]$  et  $t[j]$



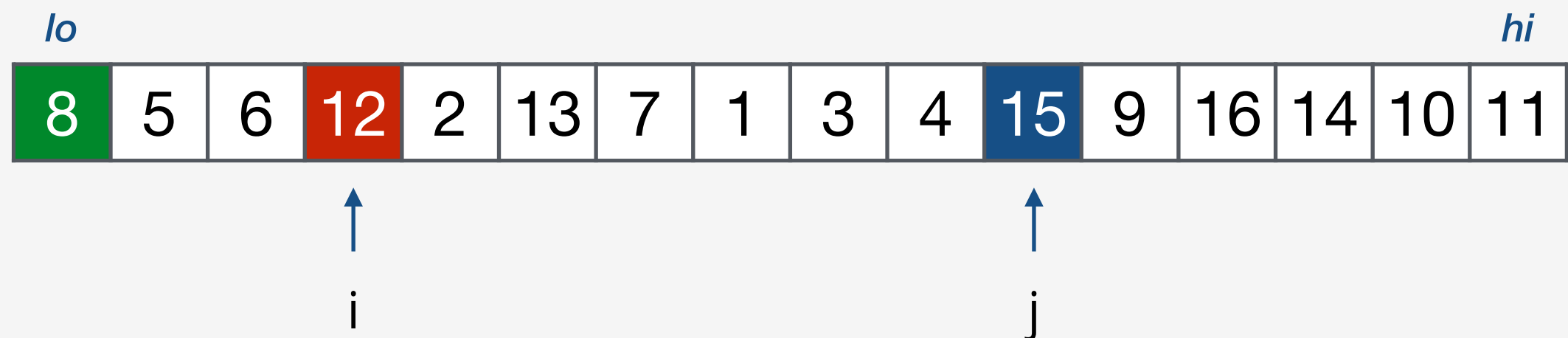
# Partitionnement

- ▶ Répéter jusqu'à ce que  $i$  et  $j$  se croisent
  - ❖ incrémenter l'indice  $i$  tant que  $t[i] < t[lo]$
  - ❖ décrémenter l'indice  $j$  tant que  $t[j] > t[lo]$
  - ❖ échanger  $t[i]$  et  $t[j]$ , incrémenter  $i$ , décrémenter  $j$
- ▶ Quand  $i$  et  $j$  se croisent
  - ❖ échanger  $t[lo]$  et  $t[j]$



# Partitionnement

- ▶ Répéter jusqu'à ce que  $i$  et  $j$  se croisent
  - ❖ incrémenter l'indice  $i$  tant que  $t[i] < t[lo]$
  - ❖ décrémenter l'indice  $j$  tant que  $t[j] > t[lo]$
  - ❖ échanger  $t[i]$  et  $t[j]$ , incrémenter  $i$ , décrémenter  $j$
- ▶ Quand  $i$  et  $j$  se croisent
  - ❖ échanger  $t[lo]$  et  $t[j]$



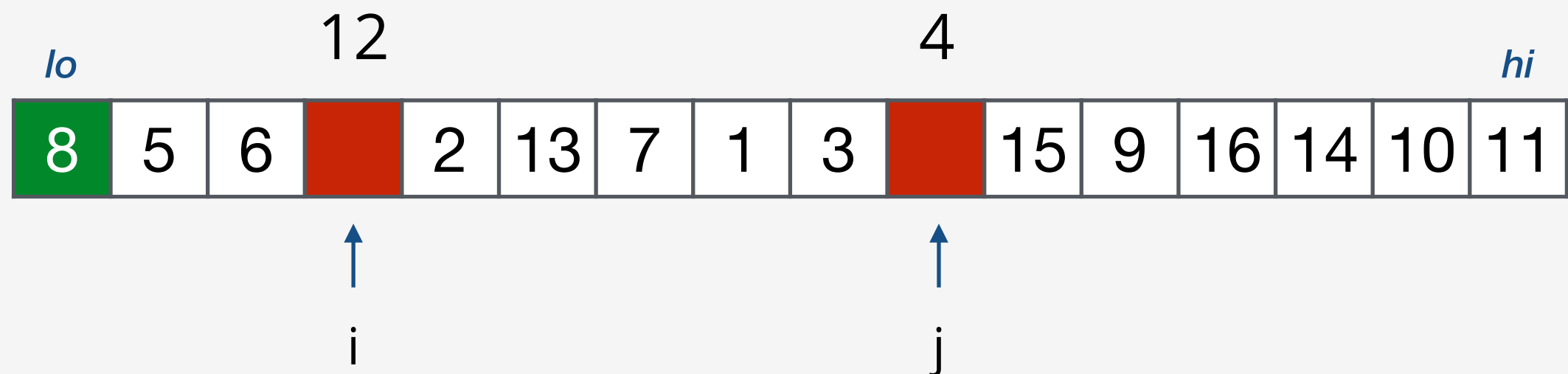
# Partitionnement

- ▶ Répéter jusqu'à ce que  $i$  et  $j$  se croisent
  - ❖ incrémenter l'indice  $i$  tant que  $t[i] < t[lo]$
  - ❖ décrémenter l'indice  $j$  tant que  $t[j] > t[lo]$
  - ❖ échanger  $t[i]$  et  $t[j]$ , incrémenter  $i$ , décrémenter  $j$
- ▶ Quand  $i$  et  $j$  se croisent
  - ❖ échanger  $t[lo]$  et  $t[j]$



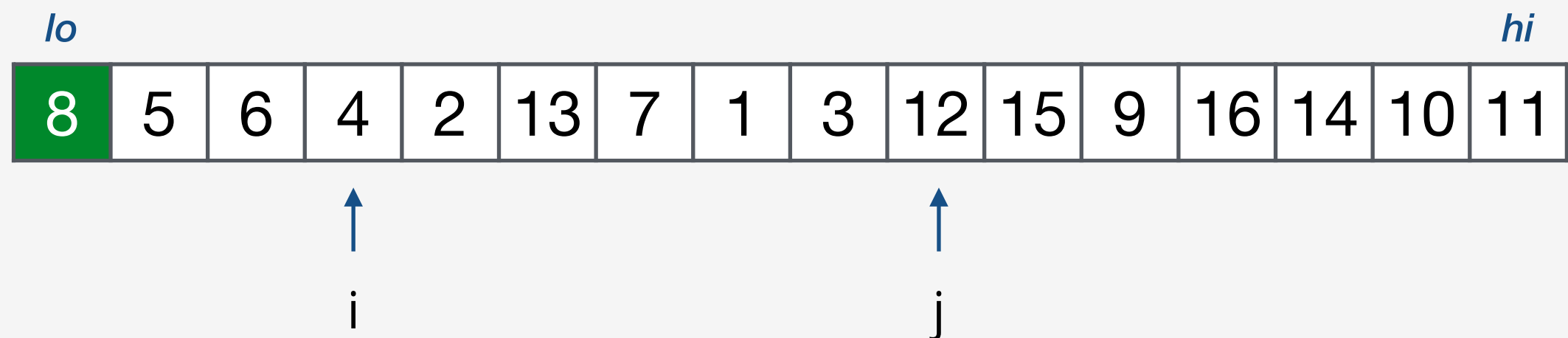
# Partitionnement

- ▶ Répéter jusqu'à ce que  $i$  et  $j$  se croisent
  - ❖ incrémenter l'indice  $i$  tant que  $t[i] < t[lo]$
  - ❖ décrémenter l'indice  $j$  tant que  $t[j] > t[lo]$
  - ❖ échanger  $t[i]$  et  $t[j]$ , incrémenter  $i$ , décrémenter  $j$
- ▶ Quand  $i$  et  $j$  se croisent
  - ❖ échanger  $t[lo]$  et  $t[j]$



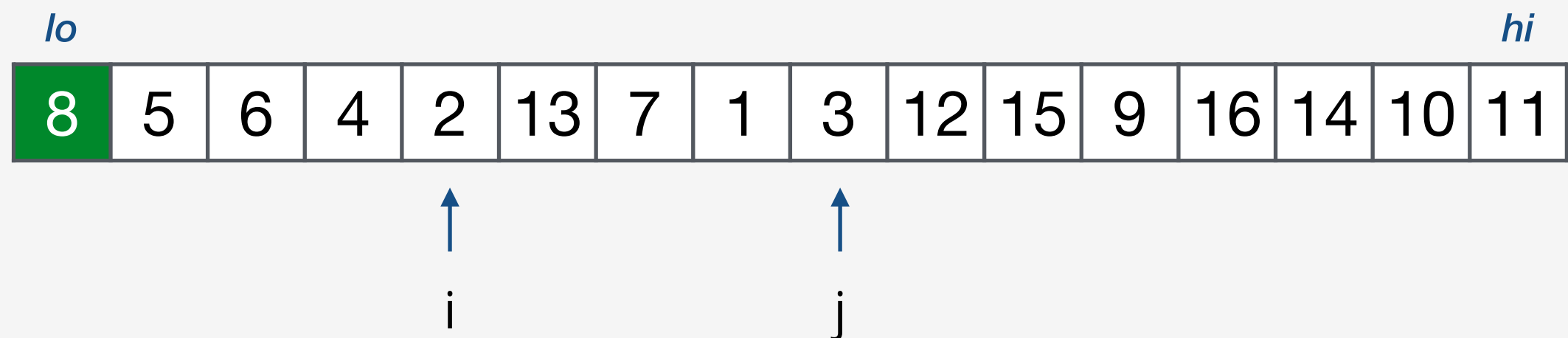
# Partitionnement

- ▶ Répéter jusqu'à ce que  $i$  et  $j$  se croisent
  - ❖ incrémenter l'indice  $i$  tant que  $t[i] < t[lo]$
  - ❖ décrémenter l'indice  $j$  tant que  $t[j] > t[lo]$
  - ❖ échanger  $t[i]$  et  $t[j]$ , incrémenter  $i$ , décrémenter  $j$
- ▶ Quand  $i$  et  $j$  se croisent
  - ❖ échanger  $t[lo]$  et  $t[j]$



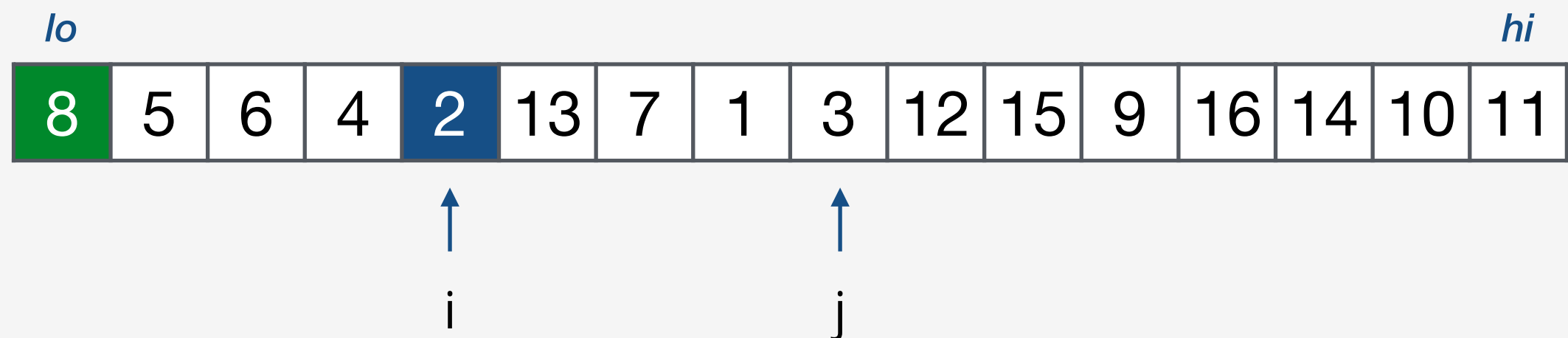
# Partitionnement

- ▶ Répéter jusqu'à ce que  $i$  et  $j$  se croisent
  - ❖ incrémenter l'indice  $i$  tant que  $t[i] < t[lo]$
  - ❖ décrémenter l'indice  $j$  tant que  $t[j] > t[lo]$
  - ❖ échanger  $t[i]$  et  $t[j]$ , incrémenter  $i$ , décrémenter  $j$
- ▶ Quand  $i$  et  $j$  se croisent
  - ❖ échanger  $t[lo]$  et  $t[j]$



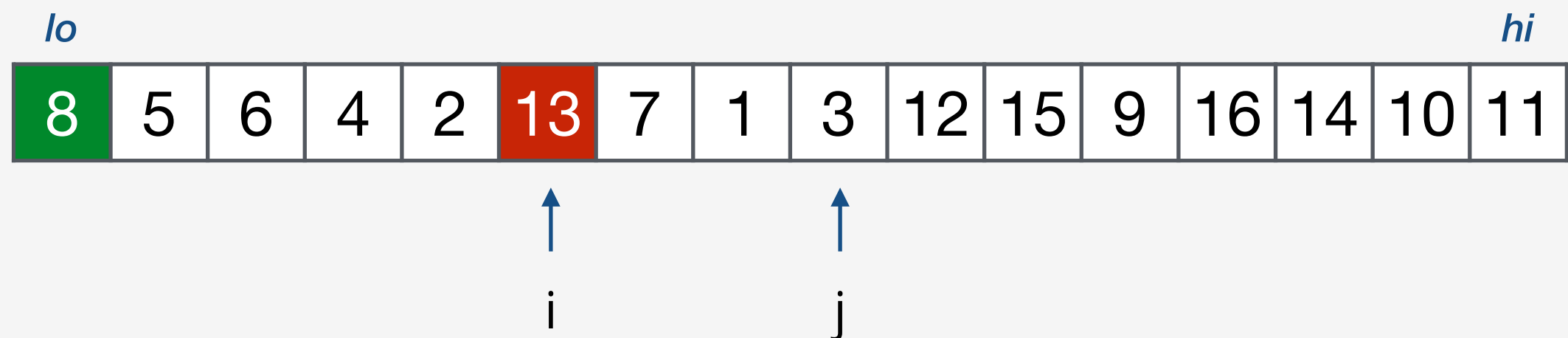
# Partitionnement

- ▶ Répéter jusqu'à ce que  $i$  et  $j$  se croisent
  - ❖ incrémenter l'indice  $i$  tant que  $t[i] < t[lo]$
  - ❖ décrémenter l'indice  $j$  tant que  $t[j] > t[lo]$
  - ❖ échanger  $t[i]$  et  $t[j]$ , incrémenter  $i$ , décrémenter  $j$
- ▶ Quand  $i$  et  $j$  se croisent
  - ❖ échanger  $t[lo]$  et  $t[j]$



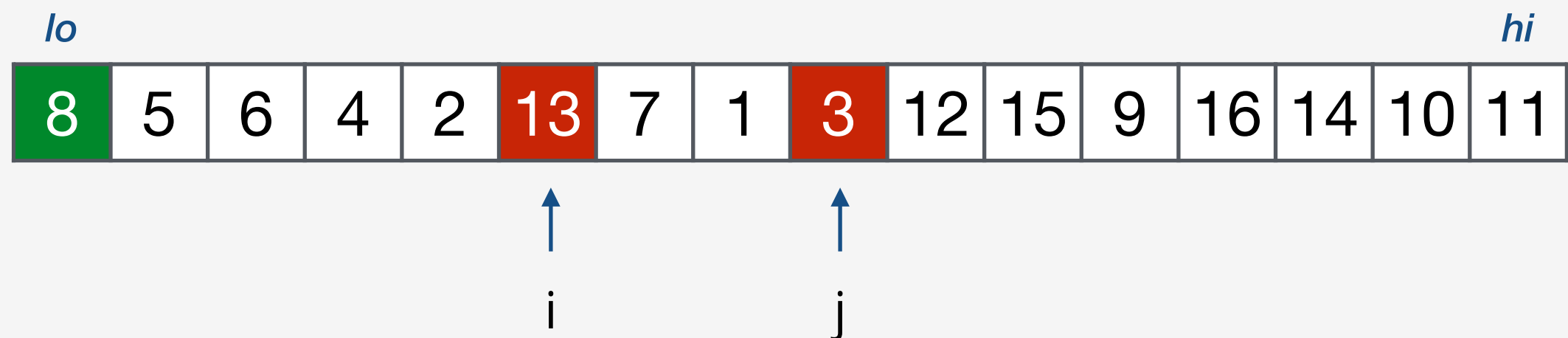
# Partitionnement

- ▶ Répéter jusqu'à ce que  $i$  et  $j$  se croisent
  - ❖ incrémenter l'indice  $i$  tant que  $t[i] < t[lo]$
  - ❖ décrémenter l'indice  $j$  tant que  $t[j] > t[lo]$
  - ❖ échanger  $t[i]$  et  $t[j]$ , incrémenter  $i$ , décrémenter  $j$
- ▶ Quand  $i$  et  $j$  se croisent
  - ❖ échanger  $t[lo]$  et  $t[j]$



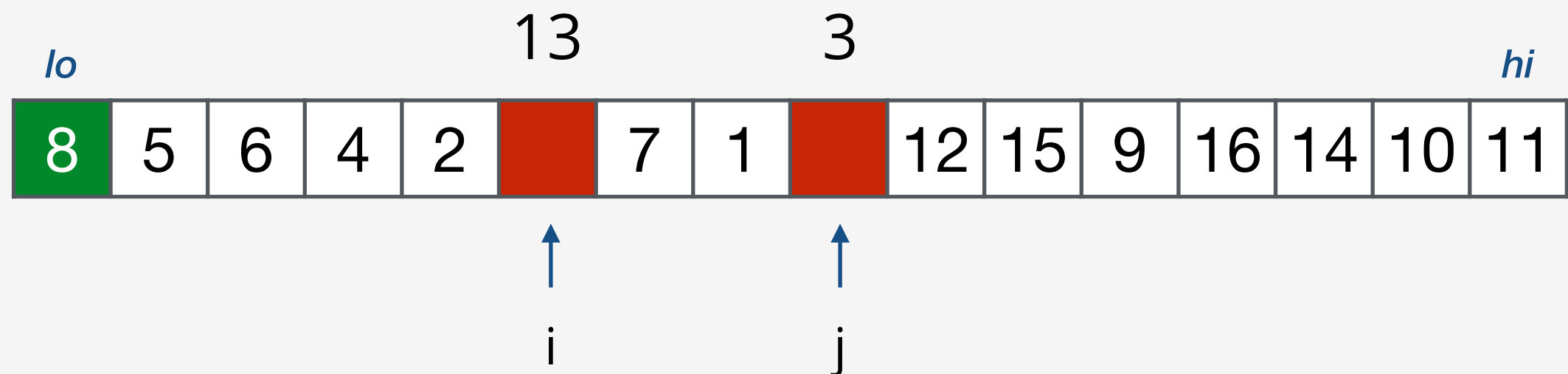
# Partitionnement

- ▶ Répéter jusqu'à ce que  $i$  et  $j$  se croisent
  - ❖ incrémenter l'indice  $i$  tant que  $t[i] < t[lo]$
  - ❖ décrémenter l'indice  $j$  tant que  $t[j] > t[lo]$
  - ❖ échanger  $t[i]$  et  $t[j]$ , incrémenter  $i$ , décrémenter  $j$
- ▶ Quand  $i$  et  $j$  se croisent
  - ❖ échanger  $t[lo]$  et  $t[j]$



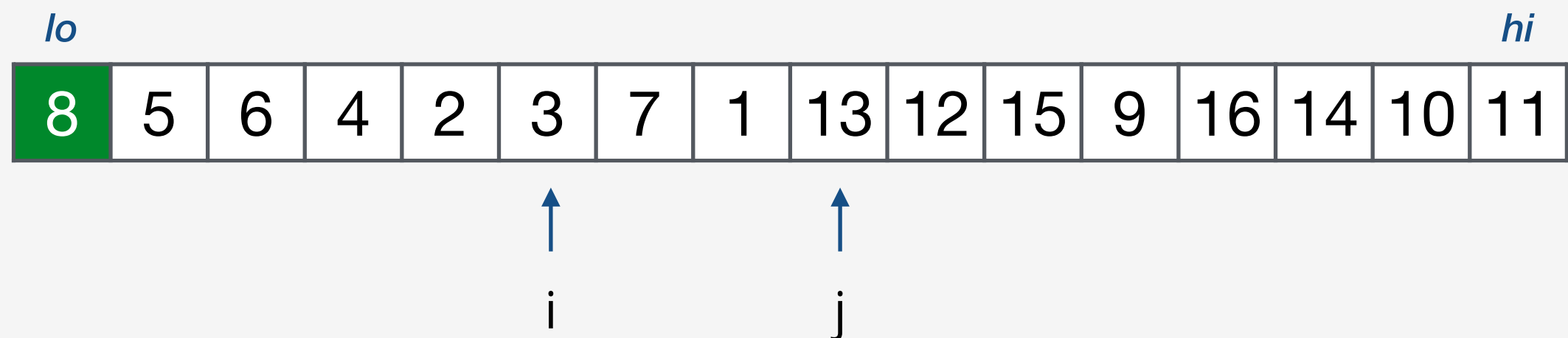
# Partitionnement

- ▶ Répéter jusqu'à ce que  $i$  et  $j$  se croisent
  - ❖ incrémenter l'indice  $i$  tant que  $t[i] < t[lo]$
  - ❖ décrémenter l'indice  $j$  tant que  $t[j] > t[lo]$
  - ❖ échanger  $t[i]$  et  $t[j]$ , incrémenter  $i$ , décrémenter  $j$
- ▶ Quand  $i$  et  $j$  se croisent
  - ❖ échanger  $t[lo]$  et  $t[j]$



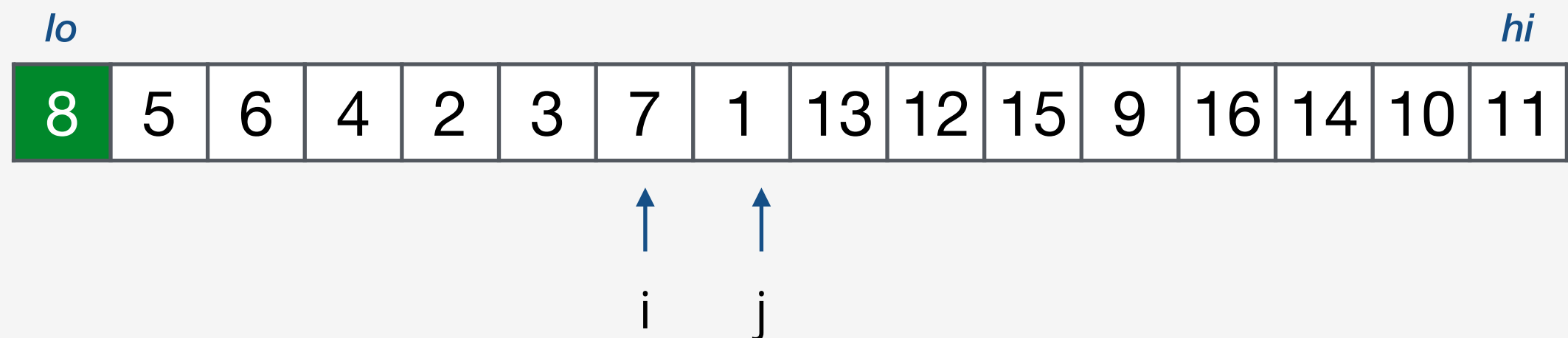
# Partitionnement

- ▶ Répéter jusqu'à ce que  $i$  et  $j$  se croisent
  - ❖ incrémenter l'indice  $i$  tant que  $t[i] < t[lo]$
  - ❖ décrémenter l'indice  $j$  tant que  $t[j] > t[lo]$
  - ❖ échanger  $t[i]$  et  $t[j]$ , incrémenter  $i$ , décrémenter  $j$
- ▶ Quand  $i$  et  $j$  se croisent
  - ❖ échanger  $t[lo]$  et  $t[j]$



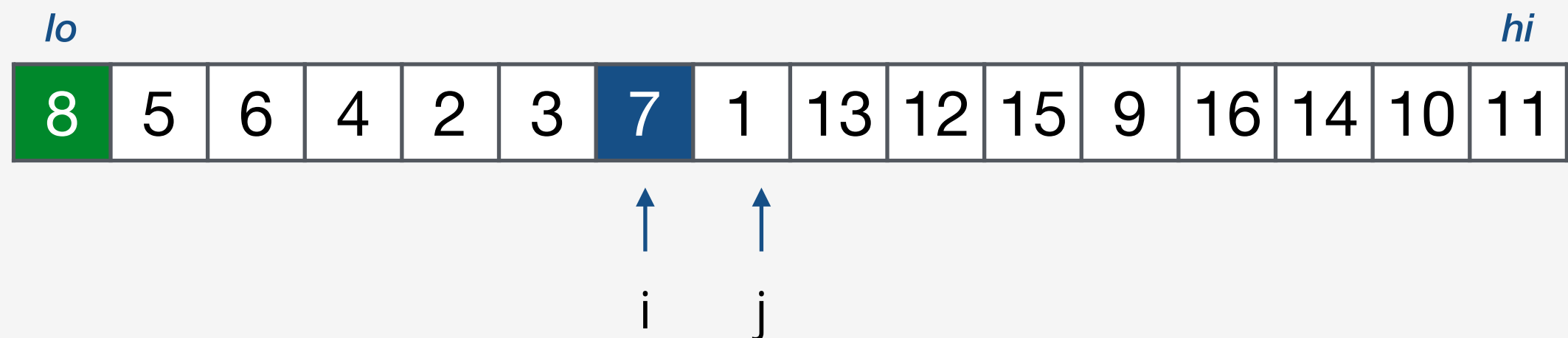
# Partitionnement

- ▶ Répéter jusqu'à ce que  $i$  et  $j$  se croisent
  - ❖ incrémenter l'indice  $i$  tant que  $t[i] < t[lo]$
  - ❖ décrémenter l'indice  $j$  tant que  $t[j] > t[lo]$
  - ❖ échanger  $t[i]$  et  $t[j]$ , incrémenter  $i$ , décrémenter  $j$
- ▶ Quand  $i$  et  $j$  se croisent
  - ❖ échanger  $t[lo]$  et  $t[j]$



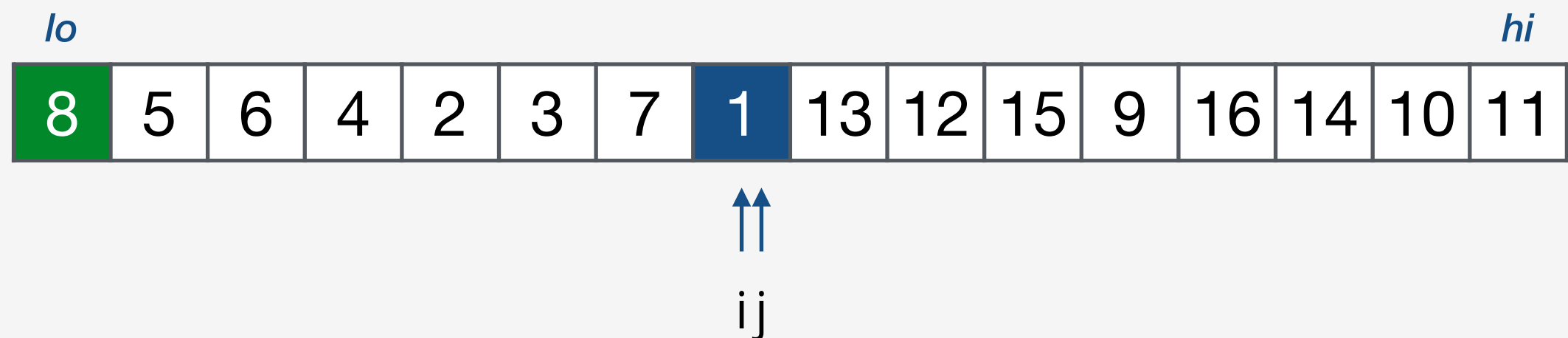
# Partitionnement

- ▶ Répéter jusqu'à ce que  $i$  et  $j$  se croisent
  - ❖ incrémenter l'indice  $i$  tant que  $t[i] < t[lo]$
  - ❖ décrémenter l'indice  $j$  tant que  $t[j] > t[lo]$
  - ❖ échanger  $t[i]$  et  $t[j]$ , incrémenter  $i$ , décrémenter  $j$
- ▶ Quand  $i$  et  $j$  se croisent
  - ❖ échanger  $t[lo]$  et  $t[j]$



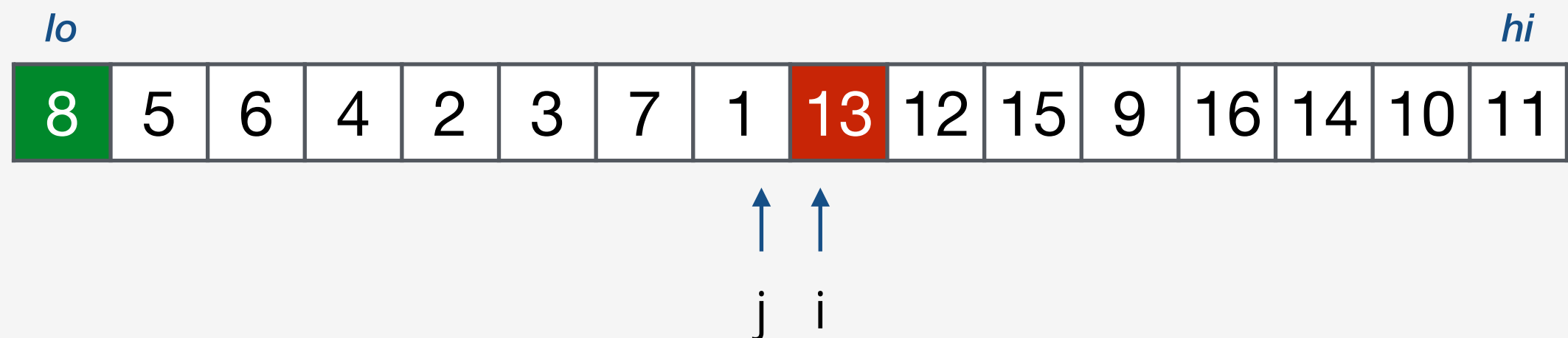
# Partitionnement

- ▶ Répéter jusqu'à ce que  $i$  et  $j$  se croisent
  - ❖ incrémenter l'indice  $i$  tant que  $t[i] < t[lo]$
  - ❖ décrémenter l'indice  $j$  tant que  $t[j] > t[lo]$
  - ❖ échanger  $t[i]$  et  $t[j]$ , incrémenter  $i$ , décrémenter  $j$
- ▶ Quand  $i$  et  $j$  se croisent
  - ❖ échanger  $t[lo]$  et  $t[j]$



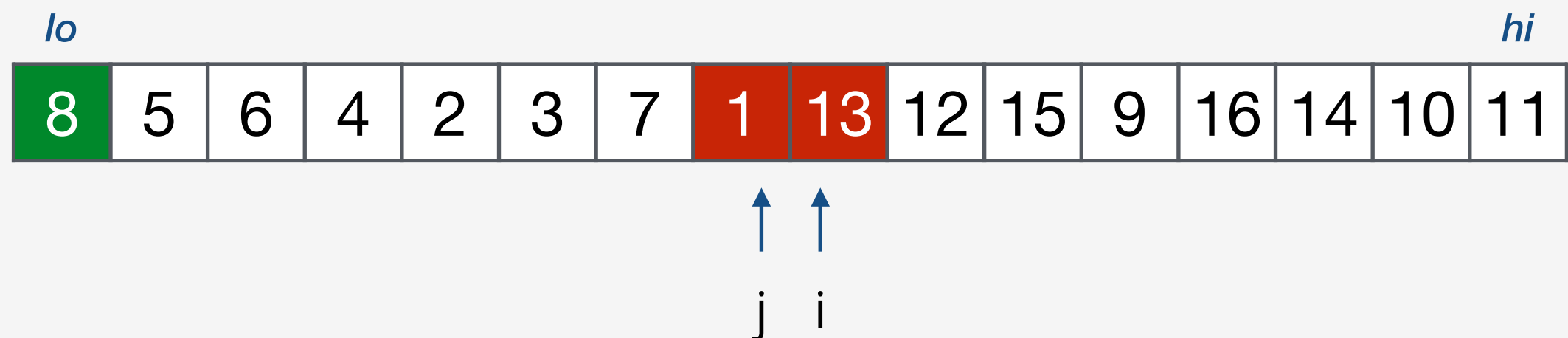
# Partitionnement

- ▶ Répéter jusqu'à ce que  $i$  et  $j$  se croisent
  - ❖ incrémenter l'indice  $i$  tant que  $t[i] < t[lo]$
  - ❖ décrémenter l'indice  $j$  tant que  $t[j] > t[lo]$
  - ❖ échanger  $t[i]$  et  $t[j]$ , incrémenter  $i$ , décrémenter  $j$
- ▶ Quand  $i$  et  $j$  se croisent
  - ❖ échanger  $t[lo]$  et  $t[j]$



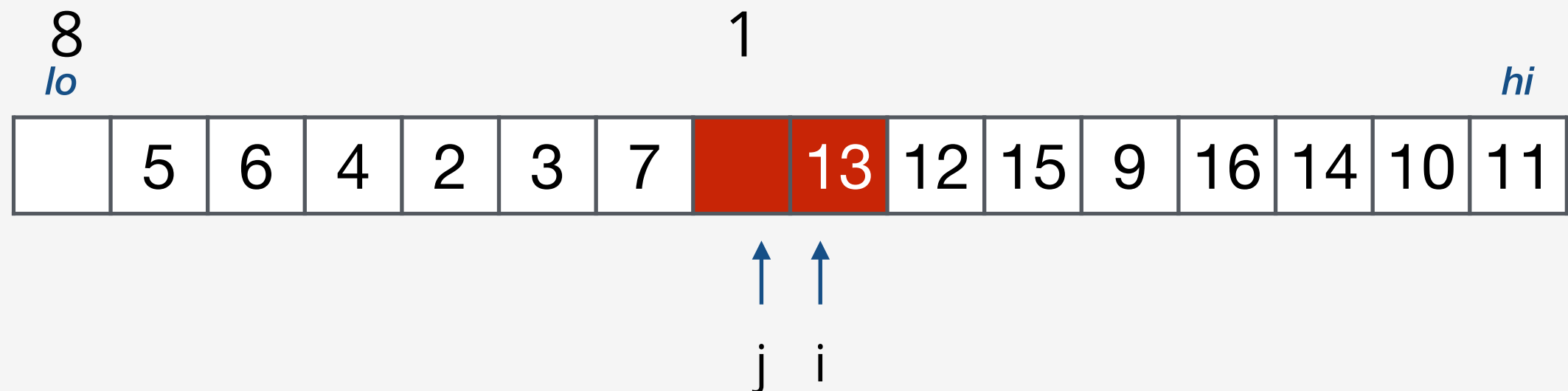
# Partitionnement

- ▶ Répéter jusqu'à ce que  $i$  et  $j$  se croisent
  - ❖ incrémenter l'indice  $i$  tant que  $t[i] < t[lo]$
  - ❖ décrémenter l'indice  $j$  tant que  $t[j] > t[lo]$
  - ❖ échanger  $t[i]$  et  $t[j]$ , incrémenter  $i$ , décrémenter  $j$
- ▶ Quand  $i$  et  $j$  se croisent
  - ❖ échanger  $t[lo]$  et  $t[j]$



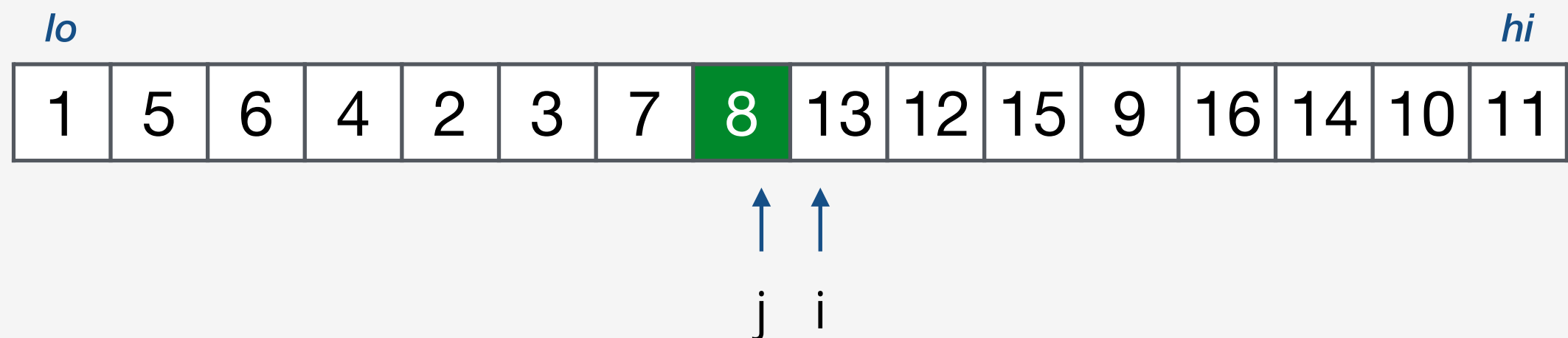
# Partitionnement

- ▶ Répéter jusqu'à ce que  $i$  et  $j$  se croisent
  - ❖ incrémenter l'indice  $i$  tant que  $t[i] < t[lo]$
  - ❖ décrémenter l'indice  $j$  tant que  $t[j] > t[lo]$
  - ❖ échanger  $t[i]$  et  $t[j]$ , incrémenter  $i$ , décrémenter  $j$
- ▶ Quand  $i$  et  $j$  se croisent
  - ❖ échanger  $t[lo]$  et  $t[j]$



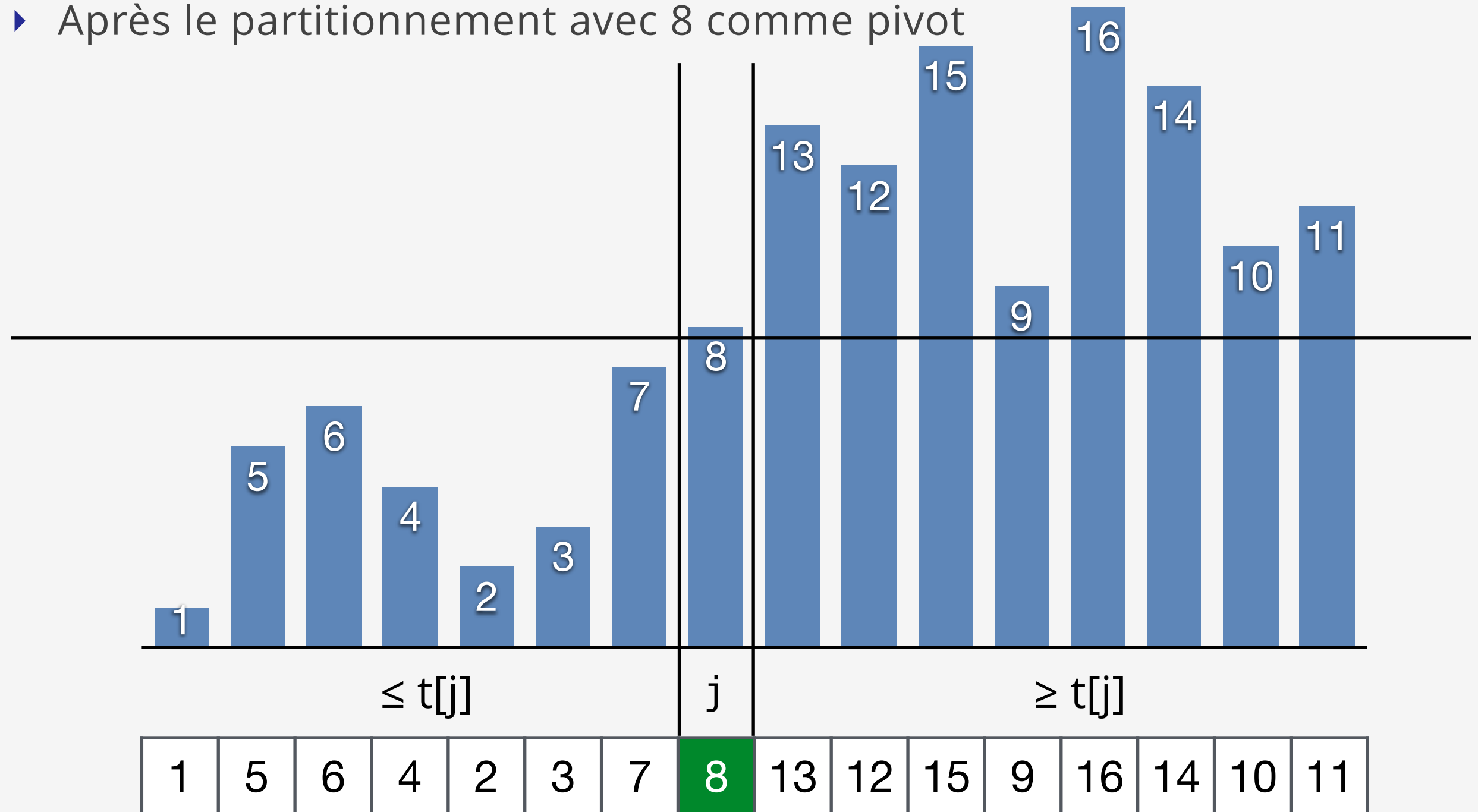
# Partitionnement

- ▶ Répéter jusqu'à ce que  $i$  et  $j$  se croisent
  - ❖ incrémenter l'indice  $i$  tant que  $t[i] < t[lo]$
  - ❖ décrémenter l'indice  $j$  tant que  $t[j] > t[lo]$
  - ❖ échanger  $t[i]$  et  $t[j]$ , incrémenter  $i$ , décrémenter  $j$
- ▶ Quand  $i$  et  $j$  se croisent
  - ❖ échanger  $t[lo]$  et  $t[j]$



# Partitionnement

- Après le partitionnement avec 8 comme pivot



# Partitionnement

- ▶ au début  $t[lo]$  contient le pivot
- ▶ la fonction renvoie l'indice du pivot après le partitionnement

```
int partition(int t[], int lo, int hi) {  
    int i = lo+1;  
    int j = hi;  
    while (1) {  
        while (less(t[i], t[lo]) && i < hi)  
            i++;  
        while (less(t[lo], t[j]) && j > lo)  
            j--;  
        if (i >= j) break;  
        swap(&t[i], &t[j]);  
        i++;  
        j--;  
    }  
    swap(&t[lo], &t[j]);  
    return j;  
}
```

$t[lo]$  est le pivot

incrémenter l'indice  $i$  tant que  $t[i] < t[lo]$

décrémenter l'indice  $j$  tant que  $t[j] > t[lo]$

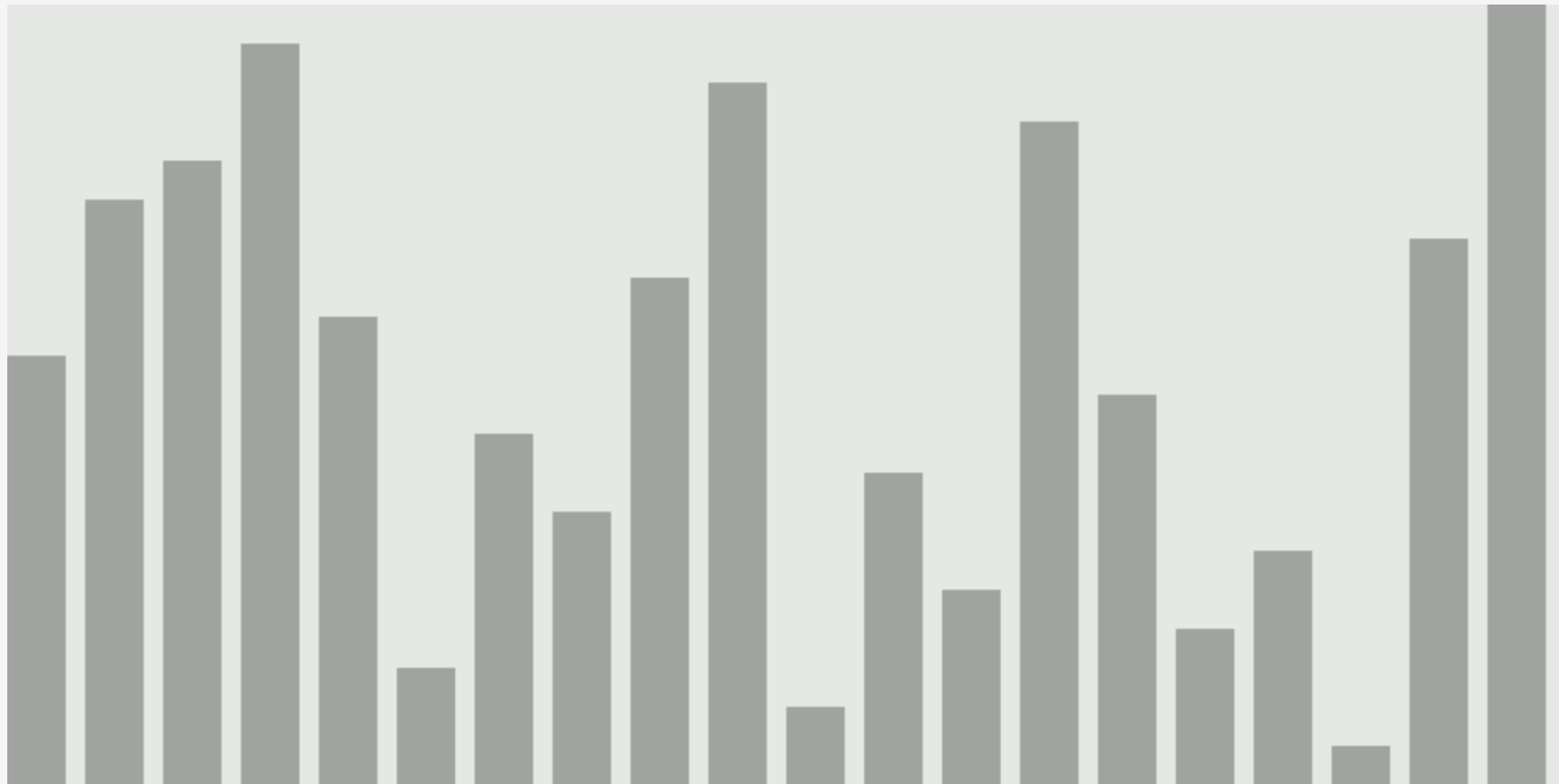
si  $i$  et  $j$  se croisent, terminer la boucle

échanger  $t[i]$  et  $t[j]$

échanger le pivot avec  $t[j]$  et renvoyer l'indice du pivot

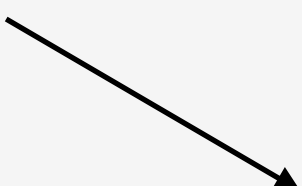
# Implantation

```
void quicksort(int t[], int lo, int hi) {  
    if (lo >= hi)  
        return;  
    int pivot = partition(t, lo, hi);  
    quicksort(t, lo, pivot-1);  
    quicksort(t, pivot+1, hi);  
}
```



# Exécution, quicksort

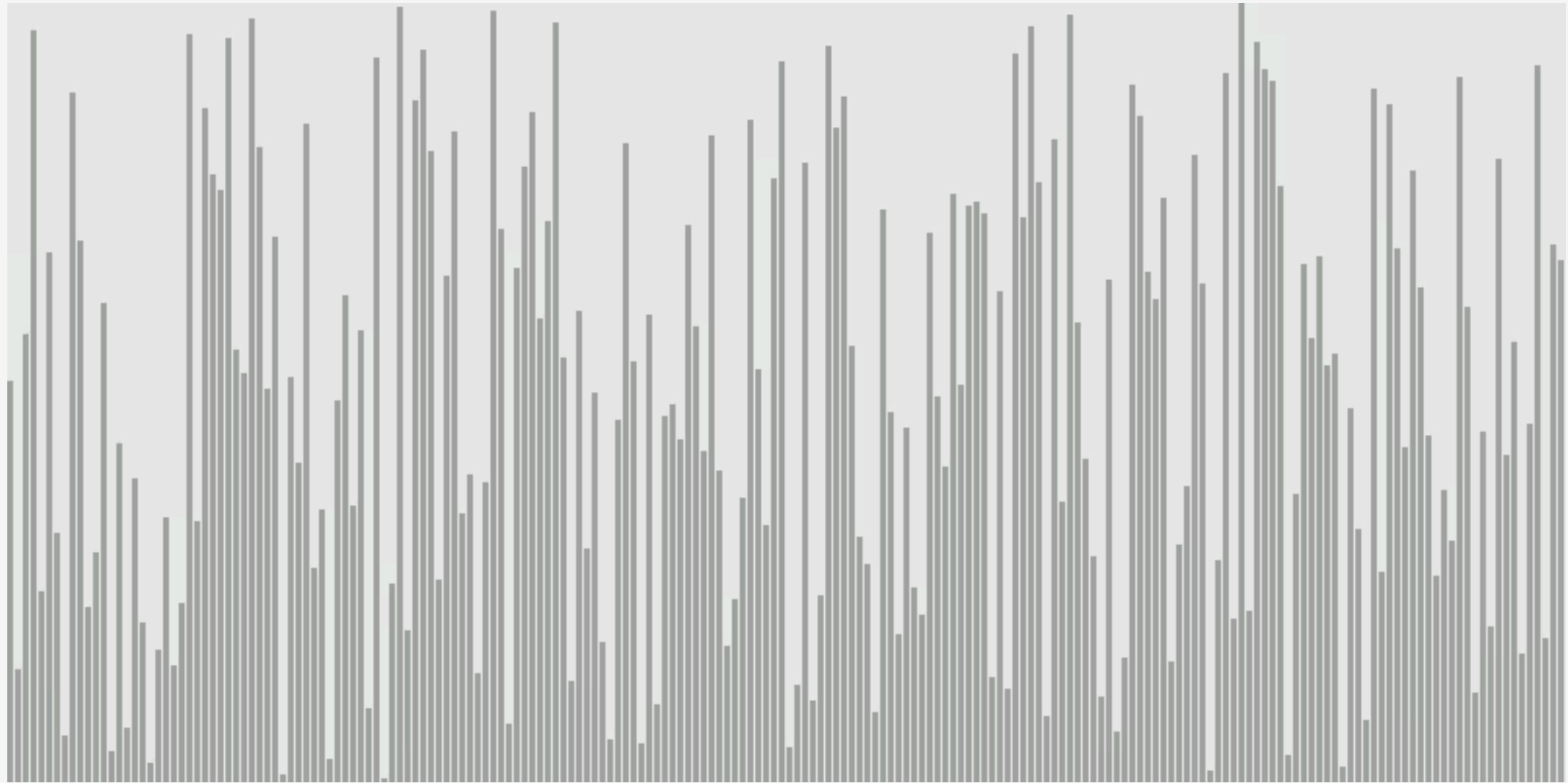
ne pas partitionner  
les tableaux de taille 1



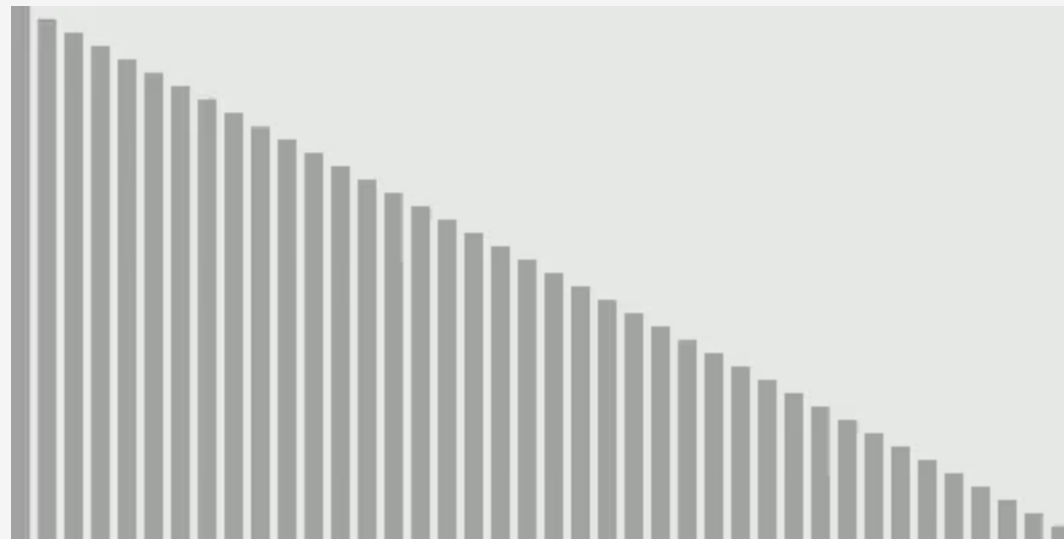
|    |   |    |   |   |   |   |   |   |   |   |   |   |
|----|---|----|---|---|---|---|---|---|---|---|---|---|
|    |   |    | t |   |   |   |   |   |   |   |   |   |
|    |   |    | 7 | 4 | 3 | 2 | 3 | 6 | 7 | 9 | 4 | 3 |
| lo | j | hi | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
| 0  | 7 | 9  | 4 | 4 | 3 | 2 | 3 | 6 | 3 | 7 | 9 | 7 |
| 0  | 4 | 6  | 3 | 3 | 3 | 2 | 4 | 6 | 4 | 7 | 9 | 7 |
| 0  | 2 | 3  | 3 | 2 | 3 | 3 | 4 | 6 | 4 | 7 | 9 | 7 |
| 0  | 1 | 1  | 2 | 3 | 3 | 3 | 4 | 6 | 4 | 7 | 9 | 7 |
| 0  |   | 0  | 2 | 3 | 3 | 3 | 4 | 6 | 4 | 7 | 9 | 7 |
| 3  |   | 3  | 2 | 3 | 3 | 3 | 4 | 6 | 4 | 7 | 9 | 7 |
| 5  | 6 | 6  | 2 | 3 | 3 | 3 | 4 | 4 | 6 | 7 | 9 | 7 |
| 5  |   | 5  | 2 | 3 | 3 | 3 | 4 | 4 | 6 | 7 | 7 | 9 |
| 8  | 9 | 9  | 2 | 3 | 3 | 3 | 4 | 4 | 6 | 7 | 7 | 9 |
| 8  |   | 8  | 2 | 3 | 3 | 3 | 4 | 4 | 6 | 7 | 7 | 9 |
|    |   |    | 2 | 3 | 3 | 3 | 4 | 4 | 6 | 7 | 7 | 9 |

après le partitionnement

# Quicksort $N = 200$

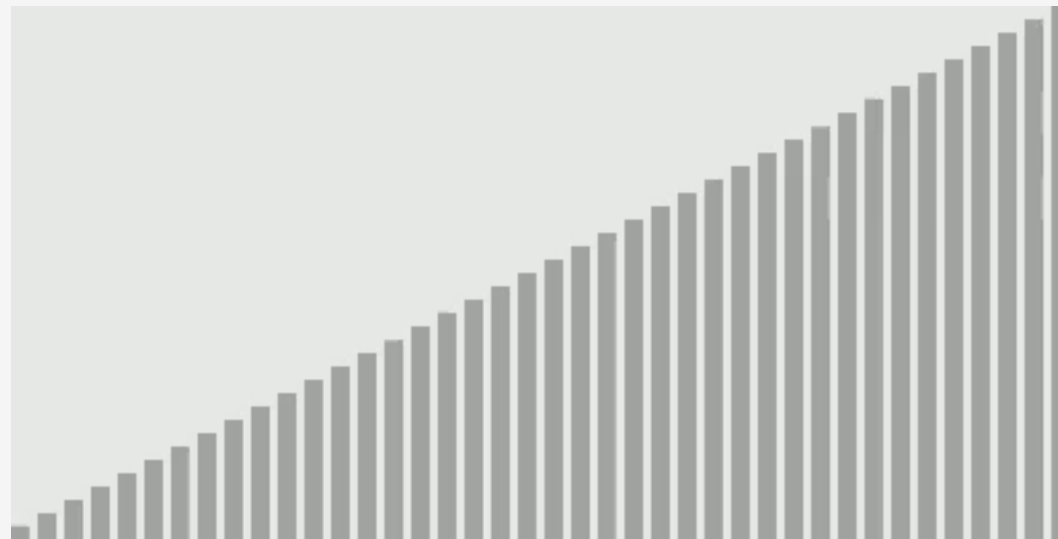


# Complexité



quelle est la complexité de quicksort appliqué à un tableau **inversé** ?

# Complexité



quelle est la complexité de quicksort appliqué à un tableau **trié** ?

# Quicksort, le pire des cas

- ▶ La complexité dans le pire des cas est de  $O(n^2)$ 
  - ❖ chaque itération décrémente la taille par 1
  - ❖ le nombre de comparaisons de l'itération  $n$  est  $n-1$
  - ❖ le nombre de comparaisons au total :  $(n-1)+(n-2)+ \dots +2+1 = (n-1) \times n/2$

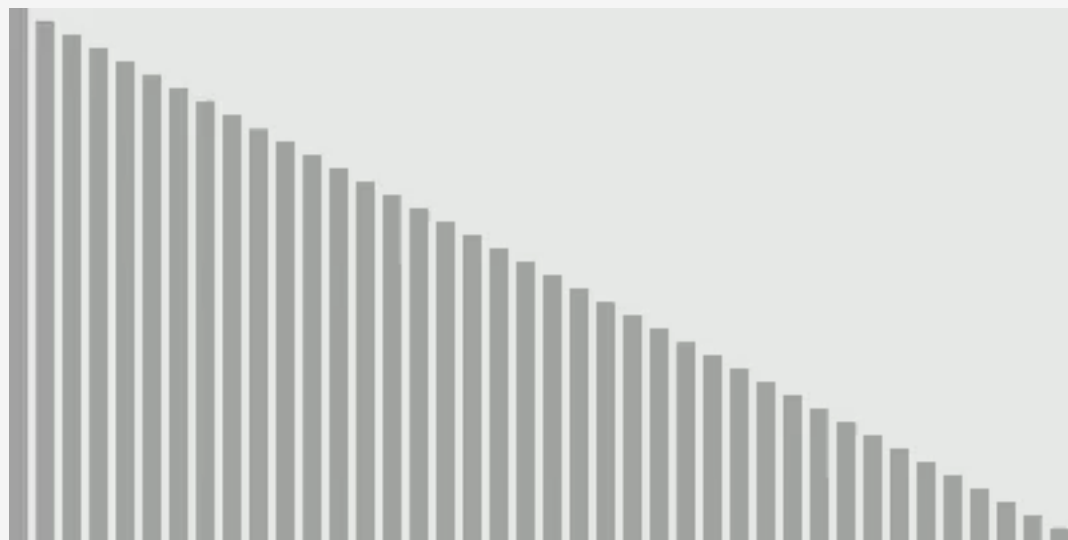


tableau inversé

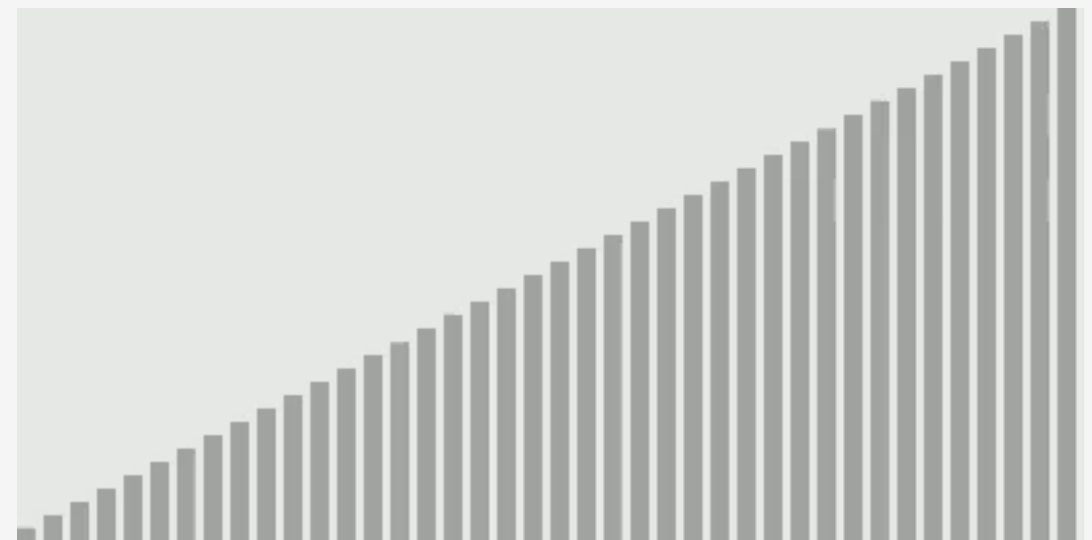
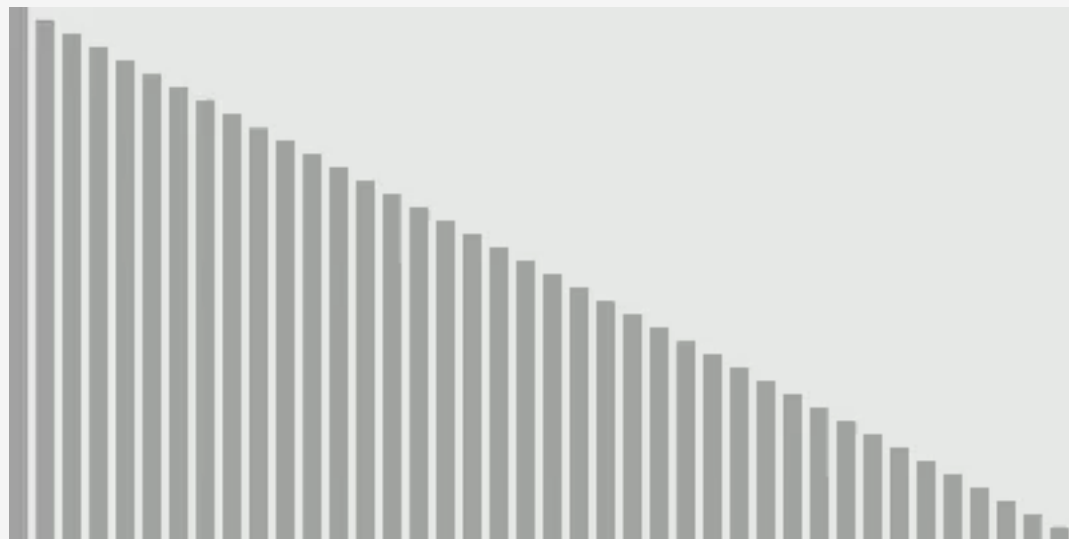


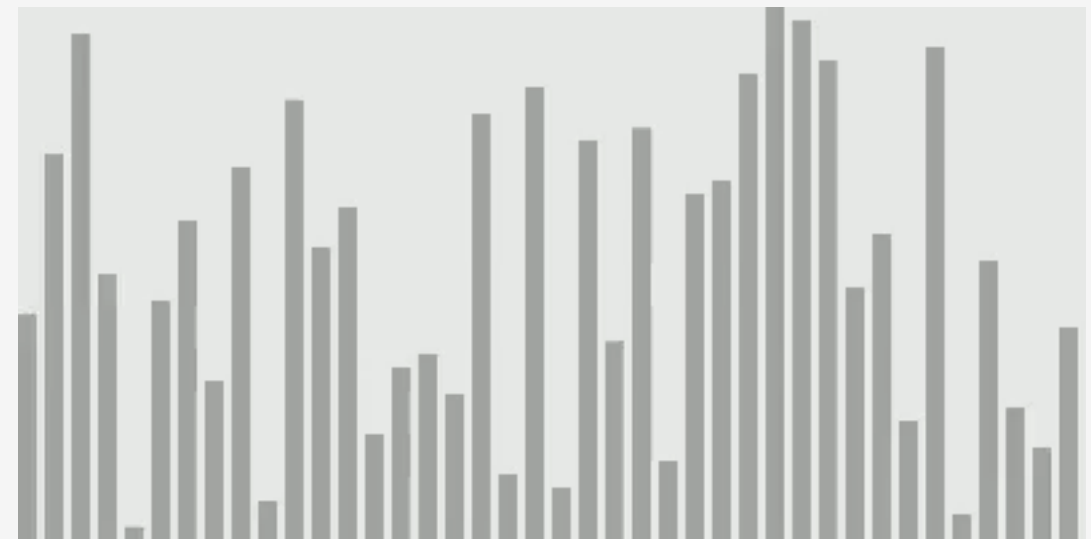
tableau trié

# Quicksort, le meilleur cas

- ▶ La complexité est de  $O(n \log_2(n))$  si le pivot du partitionnement est toujours **le médian** (voir la démonstration pour tri fusion)



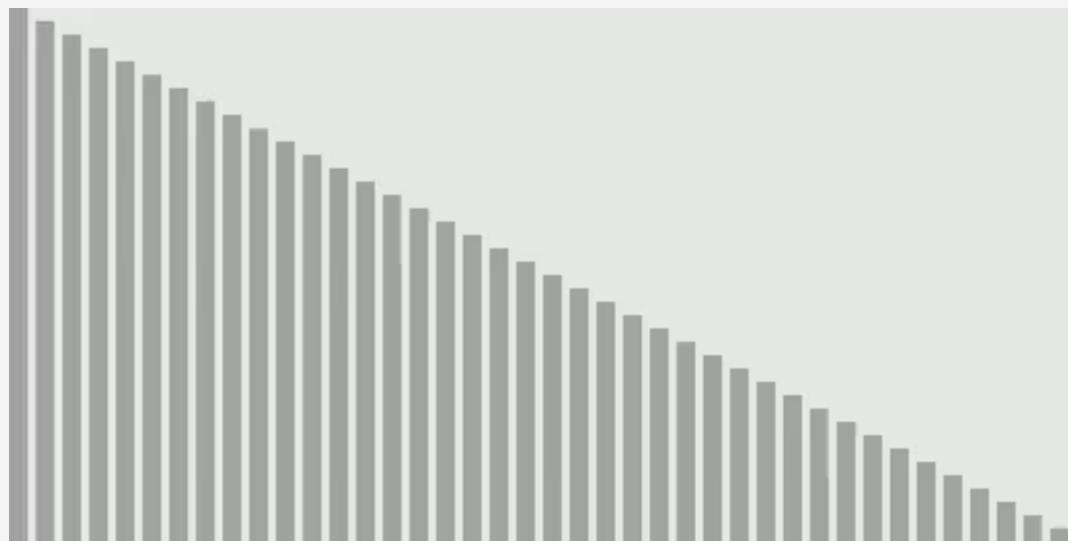
le pire de cas



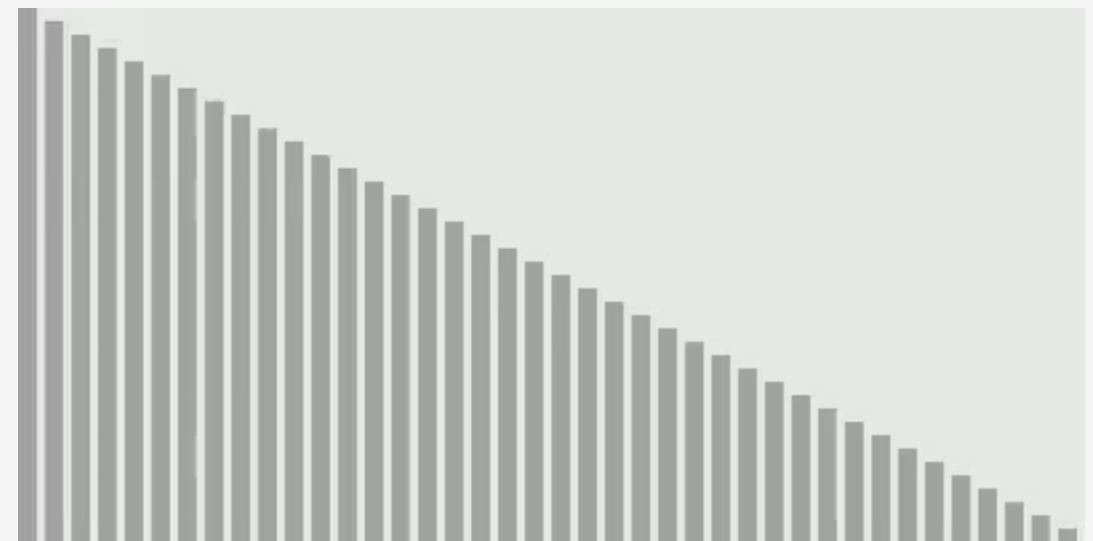
le meilleur cas

# Choix de pivot

- ▶ le vrai médian —  $O(n \log_2(n))$ 
  - ❖ on peut trouver le médian en  $O(n)$  mais trop cher en pratique
- ▶ médian de 3 :  $t[lo]$ ,  $t[lo+(hi-lo)/2]$ ,  $t[hi]$  —  $O(n^2)$ 
  - ❖ évite les problèmes des suites triées et inversées
- ▶ un indice aléatoire entre  $lo$  et  $hi$  —  $O(n \log_2(n))$  *en moyenne*
- ▶ médian de 3 éléments aléatoires —  $O(n \log_2(n))$  *en moyenne*



pivot = lo



pivot = lo+rand()%(hi-lo+1)

# Types de données

- ▶ aléatoires
- ▶ triées, inversées
- ▶ fréquence importante d'éléments ayant la même clé
  - ❖ trier des employés par rapport à l'âge
  - ❖ trier des apprentis par rapport au lieu de travail
  - ❖ cas extrêmes : 2 clés, 1 clé
- ▶ données presque triées
  - ❖ 5-10% d'éléments hors position
  - ❖ tous les éléments proches de leur position finale
  - ❖ 99% données triées + 1% données non-triées à la fin

# Améliorations quicksort

- ▶ Quicksort est inefficace (trop d'appels récur­sifs pour peu de travail) sur des petits tableaux
  - ❖ utiliser le tri par insertion si la taille du tableau qui reste est plus petite qu'une constante fixe (10—20)
- ▶ Substituer les appels récur­sifs par une pile explicite
  - ❖ taille  $\leq 64$  — pourquoi ?
- ▶ Partitionnement en 3 parties

|         |         |         |
|---------|---------|---------|
| < pivot | = pivot | > pivot |
|---------|---------|---------|

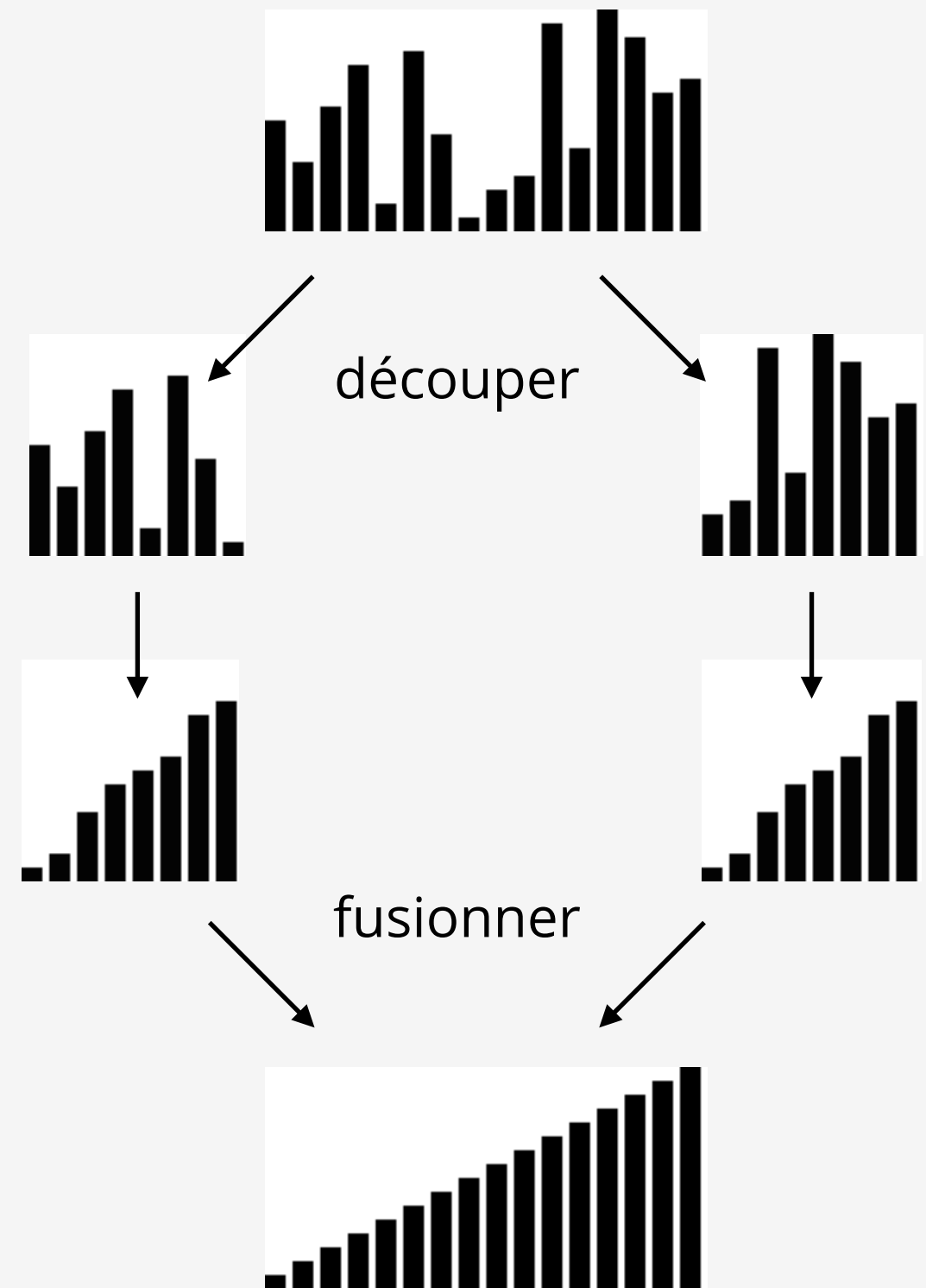
- ❖ améliore la performance en présence d'éléments ayant la même clé

# Partitionnement

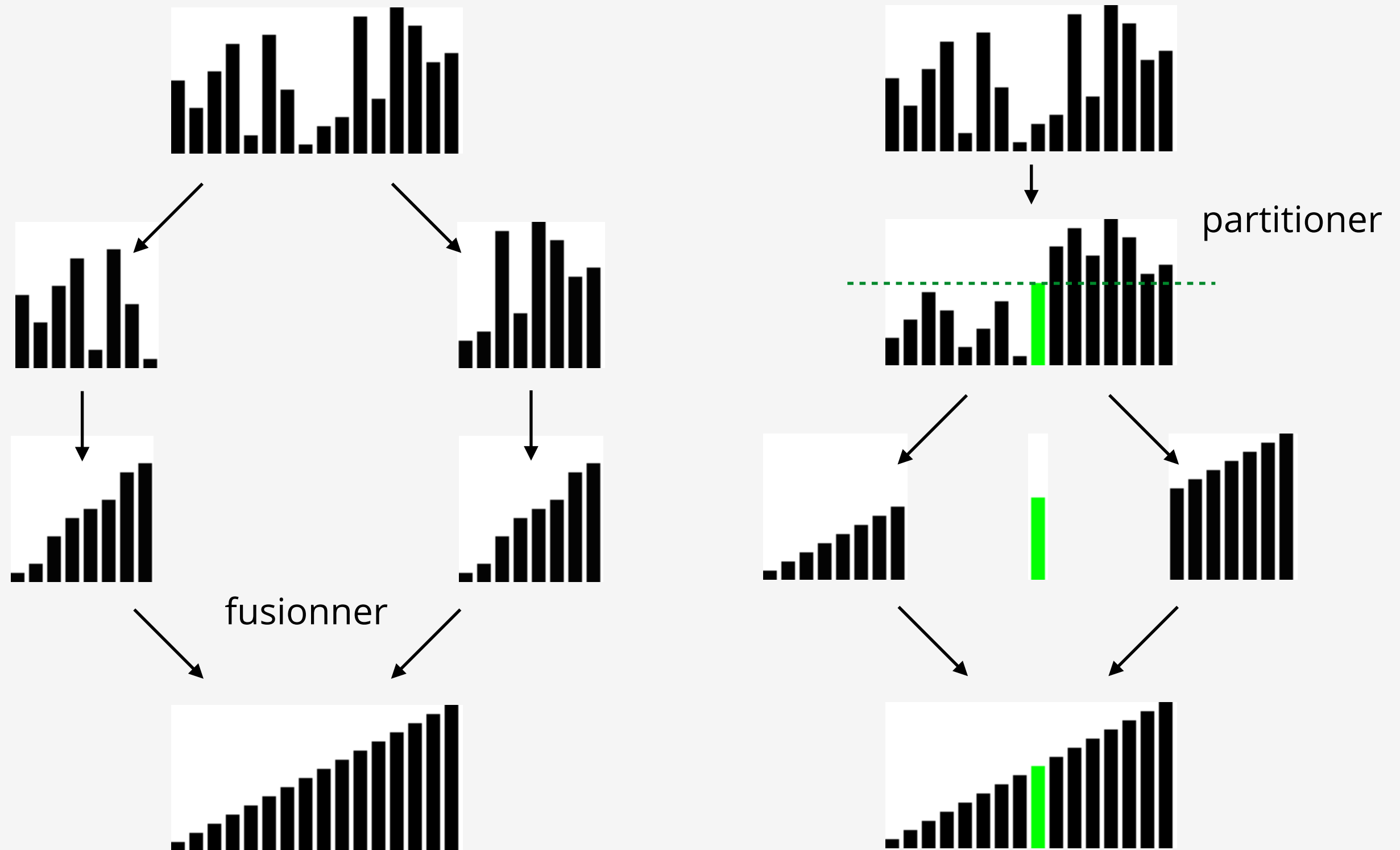
- ▶ Plusieurs algorithmes de partitionnement avec des propriétés différentes sont possibles.  
Attention ! Ce n'est pas trivial de terminer au bon moment ni de renvoyer le bon indice.
- ▶ On peut partitionner en utilisant un tableau supplémentaire.
  - ❖ C'est plus facile mais en termes de temps ca ne vaut pas le coup.
- ▶ Un algorithme de tri qui effectue tous les déplacements dans le tableau d'entrée s'appelle un **tri en place**
  - ❖ tri par sélection, tri par insertion et quicksort (avec le partitionnement donné ici) sont des tris en place

# Tri fusion (mergesort)

- ▶ Découper le tableau en deux
  - ❖  $t[lo], \dots, t[mid]$
  - ❖  $t[mid+1], \dots, t[hi]$
- ▶ Trier récursivement les deux parties
- ▶ Fusionner les deux parties triées



# Tri fusion vs quicksort



# Fusionner

```
void merge(int t[], int aux[], int lo, int mid, int hi)
```

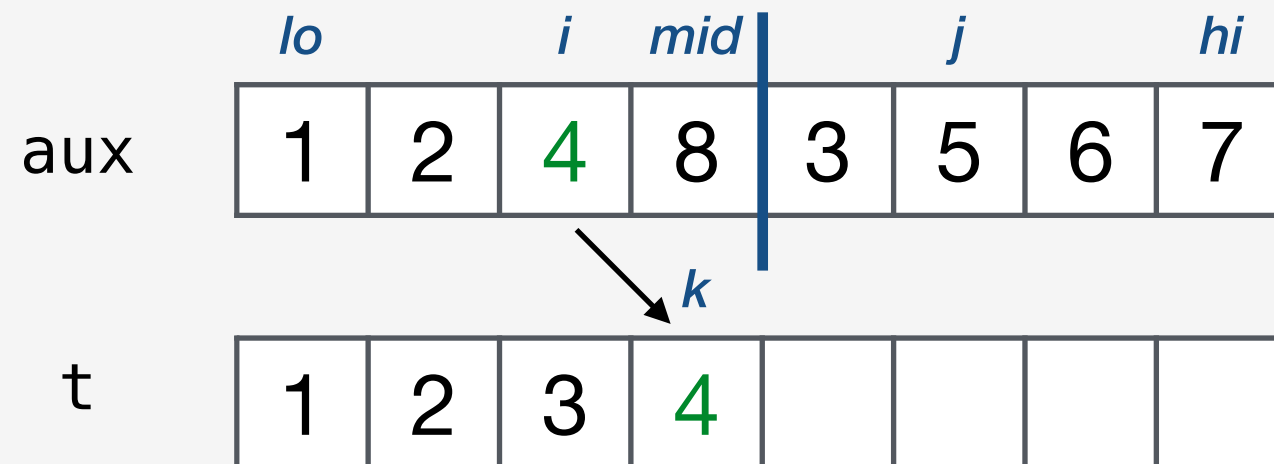
- ▶ La fonction merge (fusionner) prend en arguments
  - ❖ un tableau t
  - ❖ trois indices lo, mid, hi indiquant les deux parties triées du tableau t qu'on veut fusionner
    - ❖ t[lo] ... t[mid]
    - ❖ t[mid+1] ... t[hi]
  - ❖ un tableau auxiliaire aux pour faciliter le fusionnement
- ▶ Après la fonction, la partie t[lo] ... t[hi] est triée

|   |           |            |   |   |   |           |   |   |
|---|-----------|------------|---|---|---|-----------|---|---|
|   | <i>lo</i> | <i>mid</i> |   |   |   | <i>hi</i> |   |   |
| t | 1         | 2          | 4 | 8 | 3 | 5         | 6 | 7 |

|   |           |   |   |   |   |   |   |   |           |
|---|-----------|---|---|---|---|---|---|---|-----------|
|   | <i>lo</i> |   |   |   |   |   |   |   | <i>hi</i> |
| t | 1         | 2 | 3 | 4 | 5 | 6 | 7 | 8 |           |

# Fusionner

```
void merge(int t[], int aux[], int lo, int mid, int hi) {  
    int k;  
    for (k = lo; k <= hi; k++)  
        aux[k] = t[k];  
  
    int i = lo;  
    int j = mid+1;  
    for (k = lo; k <= hi; k++)  
        if (i > mid)  
            t[k] = aux[j++];  
        else if (j > hi)  
            t[k] = aux[i++];  
        else if (less(aux[j], aux[i]))  
            t[k] = aux[j++];  
        else  
            t[k] = aux[i++];  
}
```



# Implantation

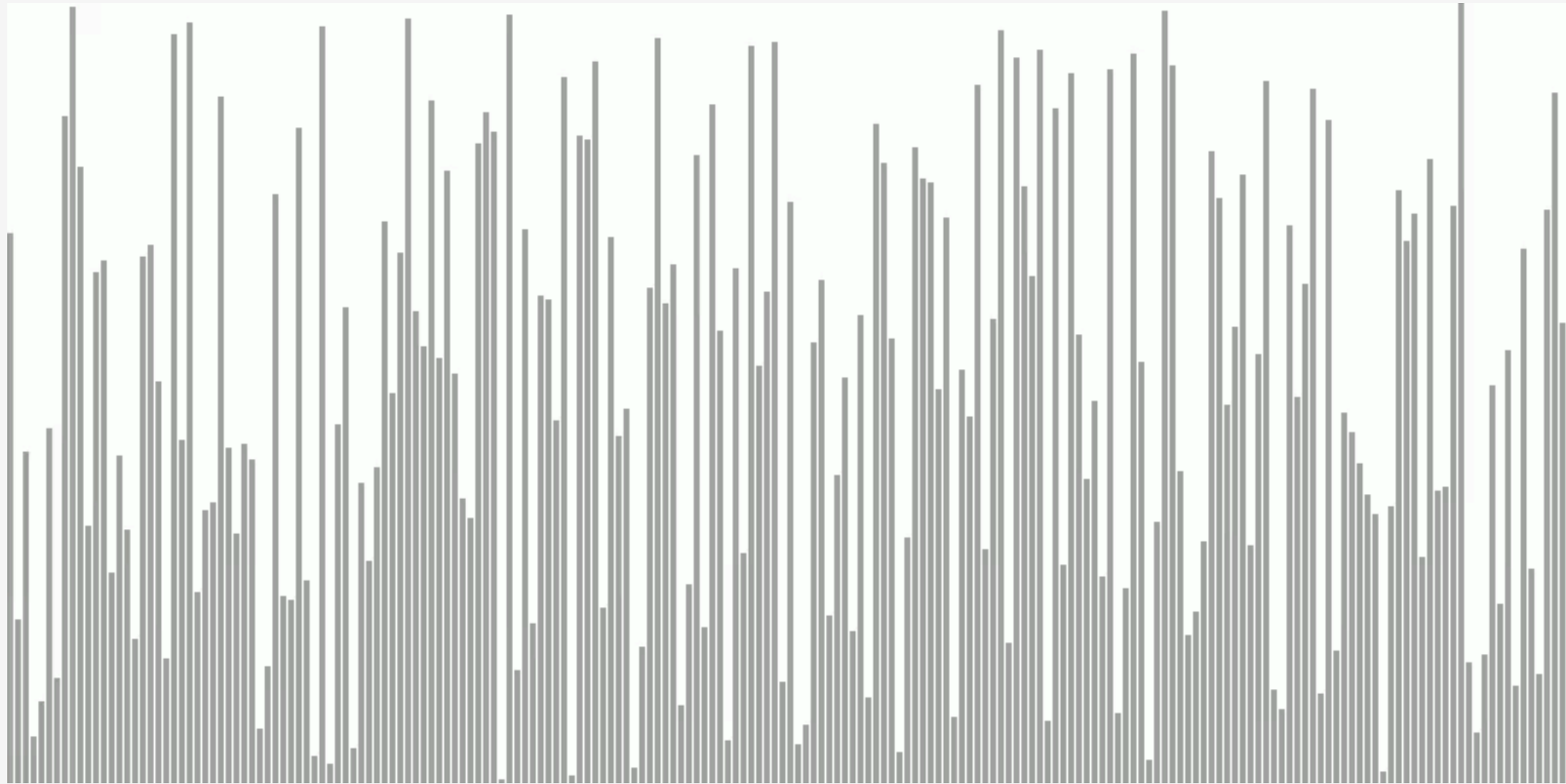
- ▶ Utiliser un seul tableau supplémentaire, aux, pour la fusion
  - ❖ aux est alloué une fois au premier appel
  - ❖ complexité d'espace supplémentaire :  $O(n)$

```
void sort(int t[], int aux[], int lo, int hi) {  
    if (lo >= hi)  
        return;  
    int mid = lo + (hi - lo) / 2;  
    sort(t, aux, lo, mid);  
    sort(t, aux, mid + 1, hi);  
    merge(t, aux, lo, mid, hi);  
}
```

```
void merge_sort(int t[], int size) {  
    int *aux = (int *)malloc(size * sizeof(int));  
    sort(t, aux, 0, size - 1);  
    free(aux);  
}
```

↑  
wrapper

# *Tri fusion* $N = 200$



# Complexité

Tri fusion utilise au plus  $n \log_2(n)$  comparaisons

- ▶ Démonstration
  - ❖ soit  $C(n)$  le nombre de comparaisons
  - ❖ supposer que  $n = 2^k$ , donc  $k = \log_2(n)$

relation de récurrence :

$$C(1) = 0$$

$$C(n) \leq 2 \times C(n/2) + n, \text{ pour } n > 1$$

comparaisons

$$n = n$$

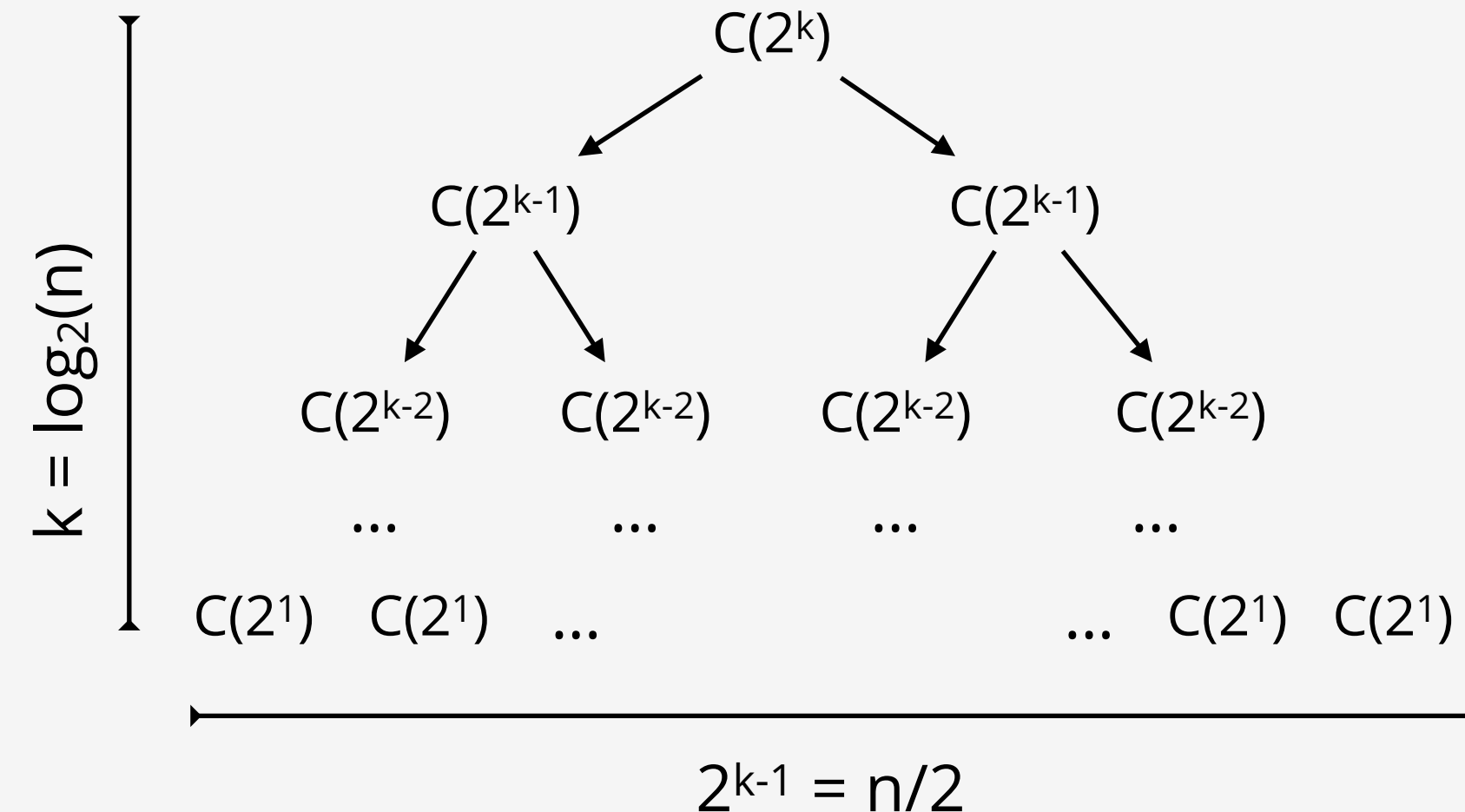
$$2 \times (n/2) = n$$

$$4 \times (n/4) = n$$

$$2^{k-1} \times (n/2^{k-1}) = n$$

---


$$\text{total} = n \log_2(n)$$



# Optimalité

- ▶ Tri fusion utilise **au plus**  $\approx n \log_2(n)$  comparaisons.
- ▶ Supposer qu'on ne peut pas accéder aux clés sauf par l'opération **less**.
  - ❖ c'est le cas pour des algorithmes génériques de C, C++ et Java qui doivent fonctionner pour n'importe quel type de clés
- ▶ Dans ce cas, on peut démontrer qu'un algorithme qui peut trier **n** clés **utilise au moins**  $\approx n \log_2(n)$  comparaisons dans le pire des cas.
- ▶ Tri fusion est donc **optimal** en termes de comparaisons dans le pire des cas.
  - ❖ **Attention !** Cela ne veut pas dire qu'il est toujours le plus rapide !

# Améliorations tri fusion

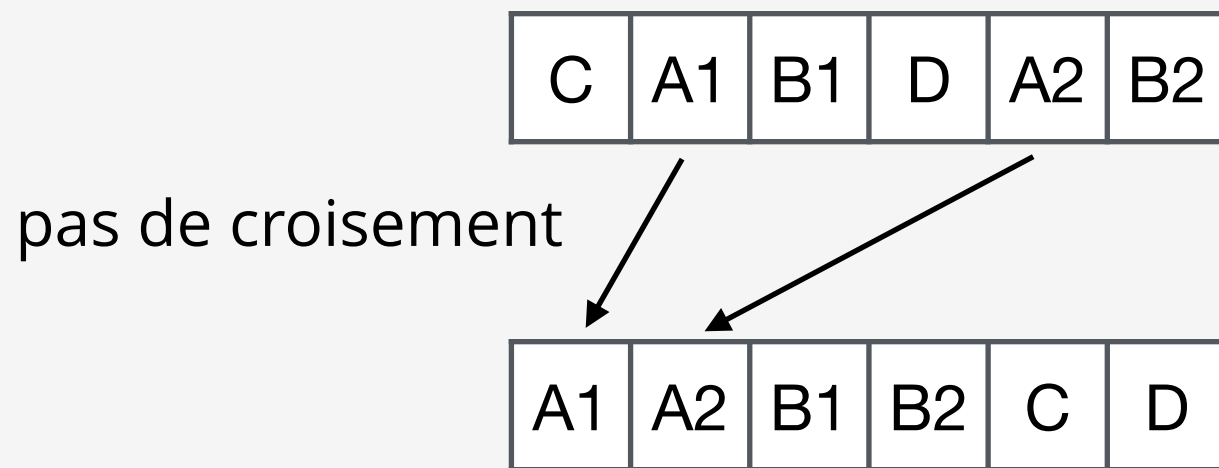
- ▶ Comme quicksort, le tri fusion est inefficace sur des petits tableaux
  - ❖ utiliser le tri par insertion si la taille du tableau qui reste  $\leq 10-20$
- ▶ Arrêter si déjà trié avant la fusion :  $t[\text{mid}] \leq t[\text{mid}+1]$
- ▶ Éviter de copier le tableau  $t$  vers  $\text{aux}$  dans la fonction **merge**
  - ❖ fait gagner du temps mais n'évite pas le tableau auxiliaire

```
void sort(int t[], int aux[], int lo, int hi) {  
    if (lo >= hi)  
        return;  
    int mid = lo+(hi-lo)/2;  
    sort(aux, t, lo, mid);  
    sort(aux, t, mid+1, hi);  
    merge(t, aux, lo, mid, hi);  
}
```

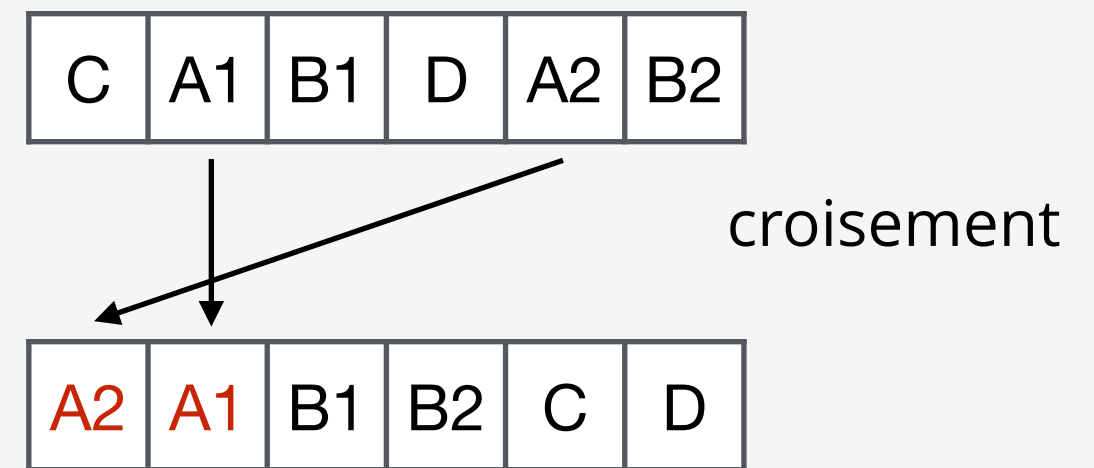
```
void merge(int t[], int aux[], int lo, int mid, int hi) {  
    int k;  
    for (k = lo; k <= hi; k++)  
        aux[k] = t[k];  
    ...  
}
```

# Stabilité

- ▶ Un algorithme de tri est **stable** s'il ne change jamais l'ordre entre données ayant la même clé
  - ❖ Exemple : trier par prénom, ensuite trier par nom
- ▶ Clé : lettre, donnée supplémentaire : chiffre

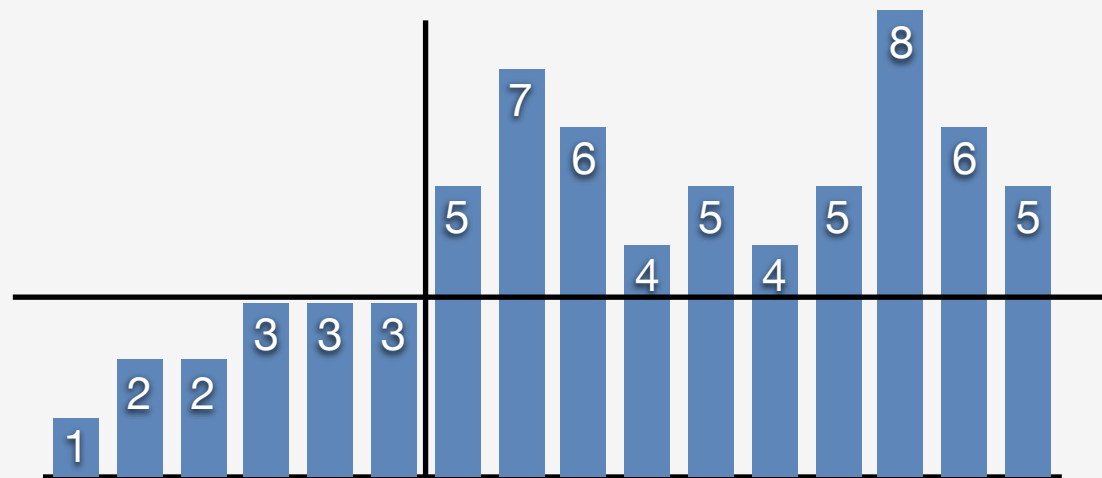


stable

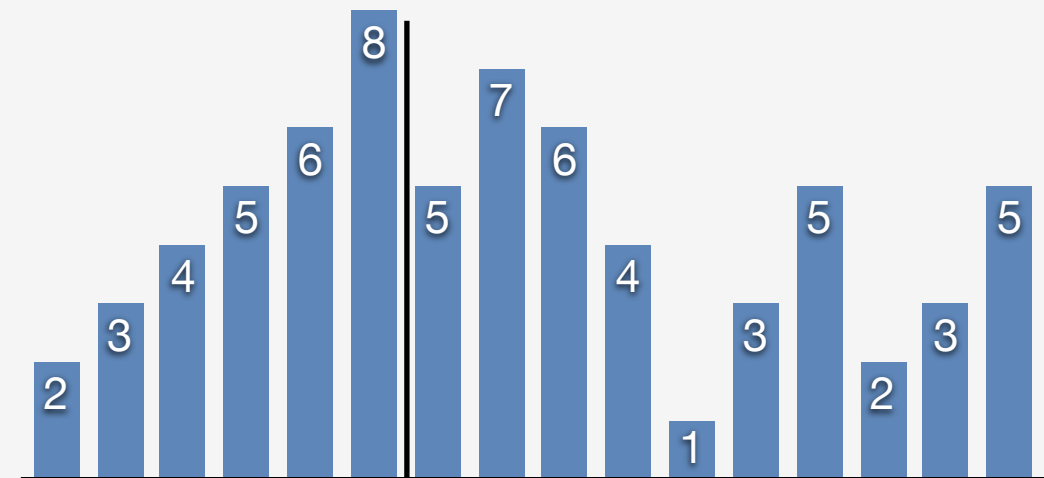


pas stable

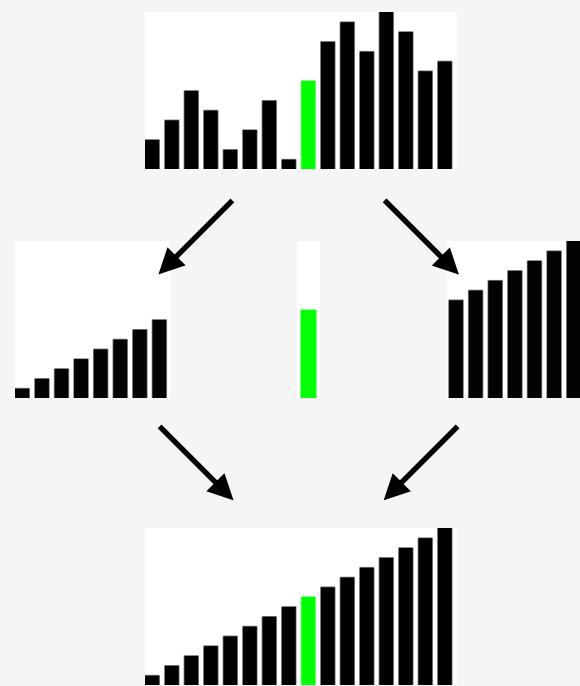
# Quels algorithmes sont stables ?



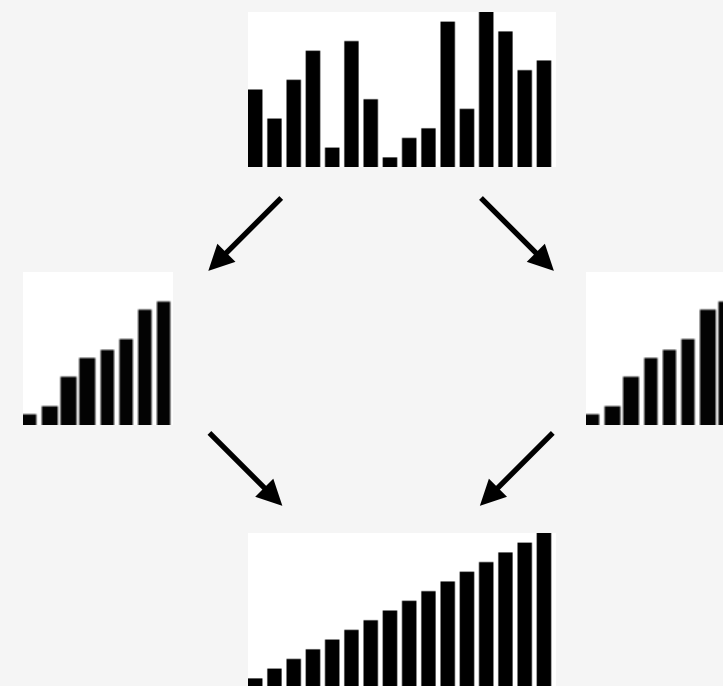
tri par sélection



tri par insertion

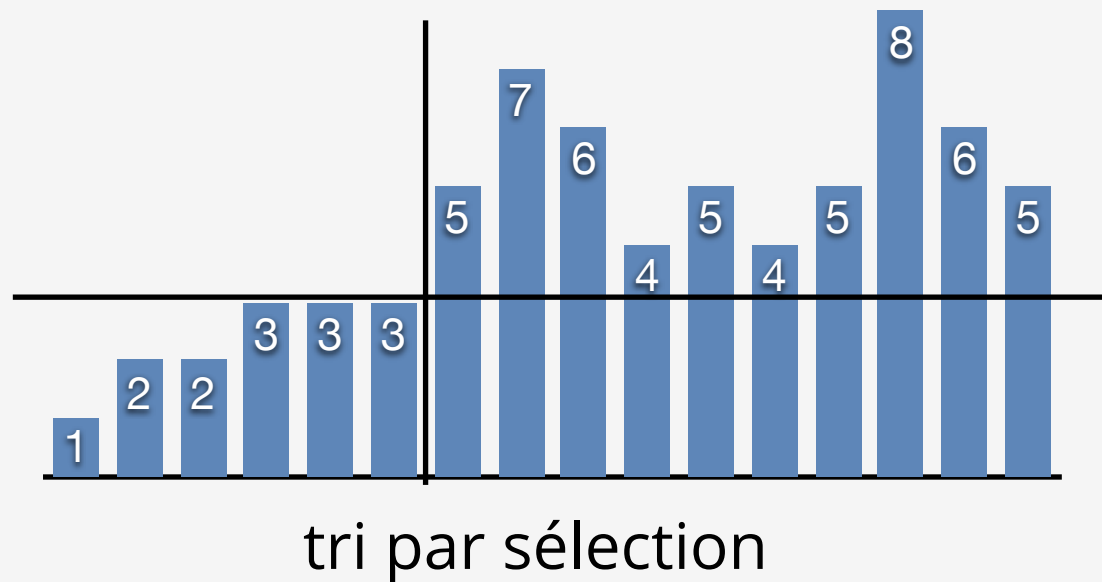


quicksort

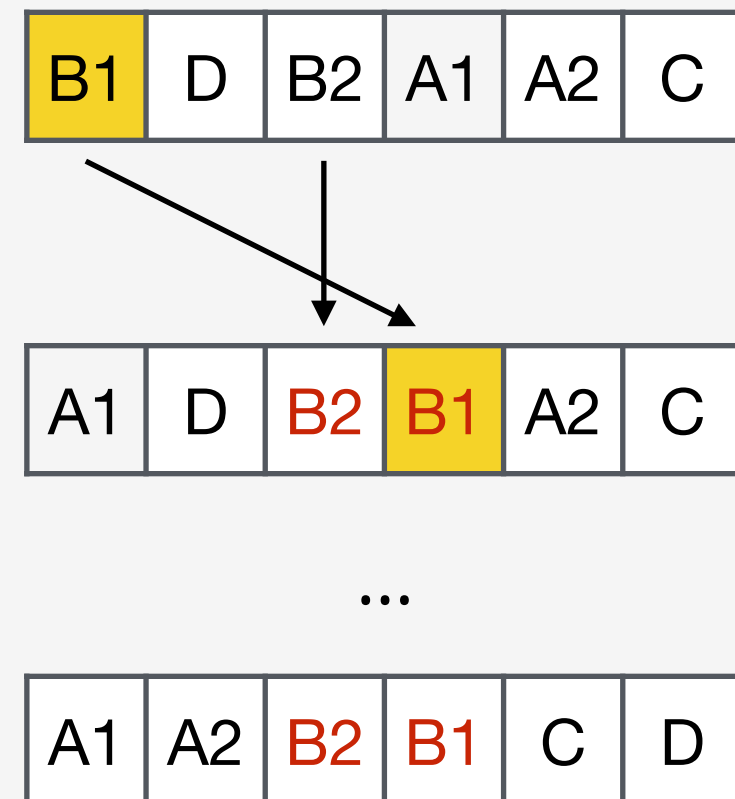


tri fusion

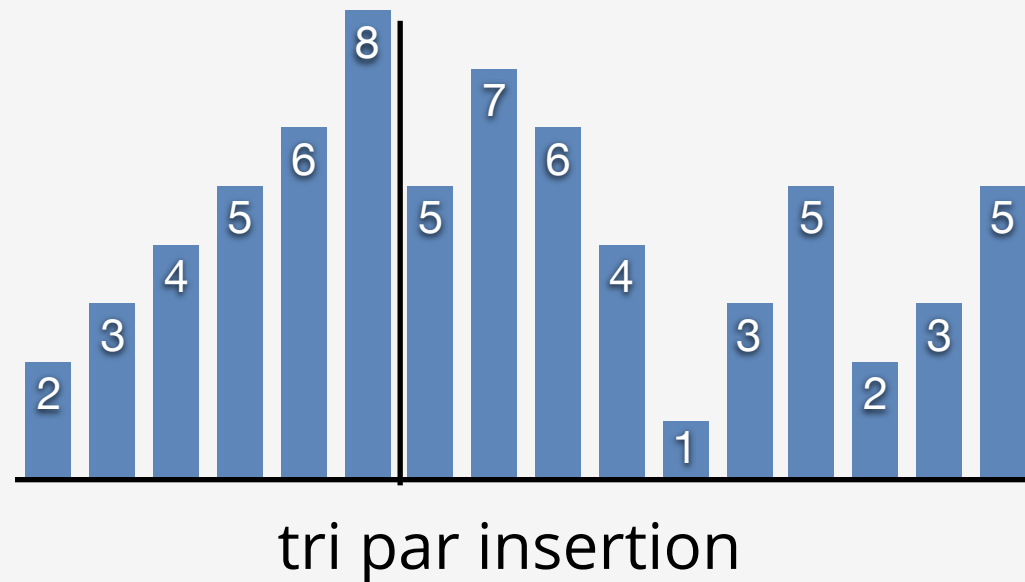
# Quels algorithmes sont stables ?



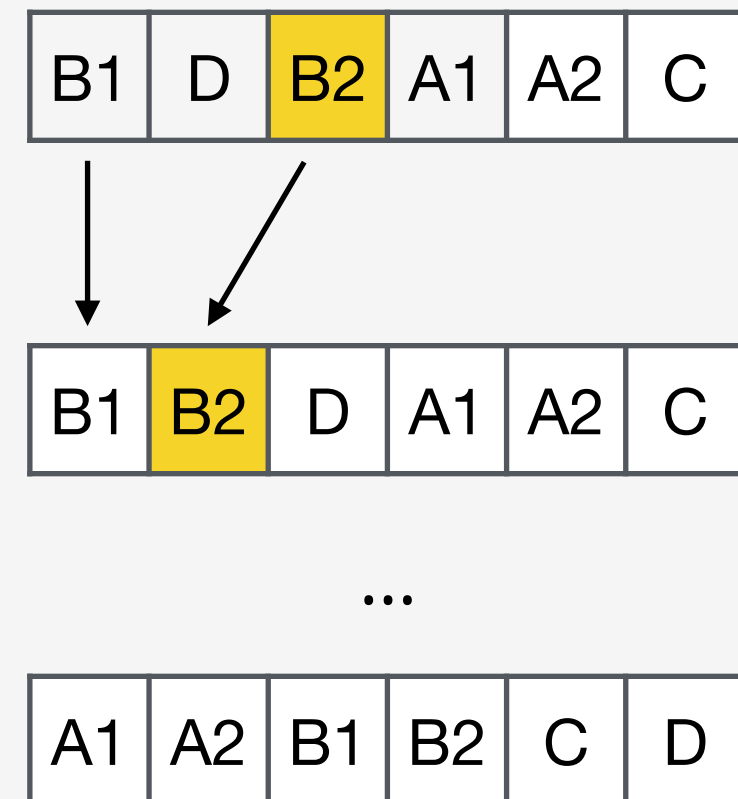
tri par sélection **n'est pas stable**



# Quels algorithmes sont stables ?

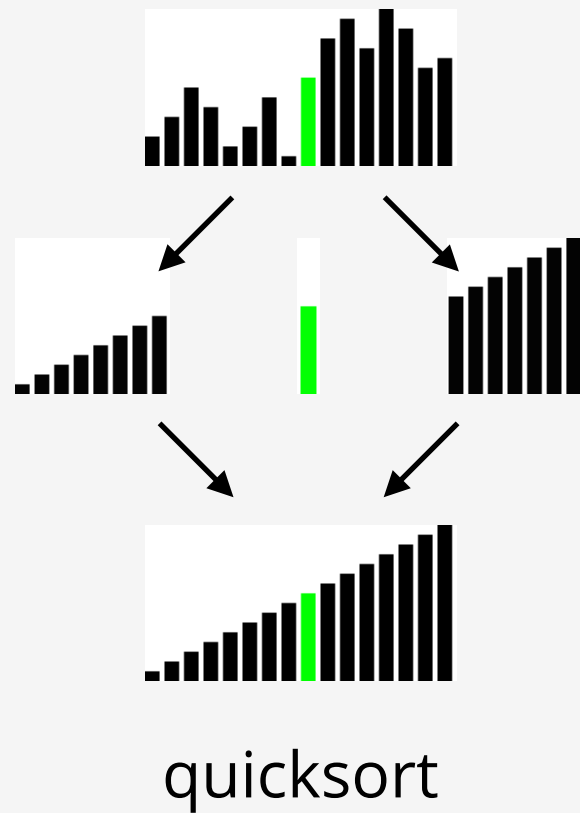


tri par insertion est **stable**



l'algorithme n'échange que des éléments voisins qui sont différents  
⇒ éléments avec les mêmes clés restent dans le même ordre

# Quels algorithmes sont stables ?



quicksort n'est pas stable

car le partitionnement  
n'est pas stable

|    |   |   |    |    |    |
|----|---|---|----|----|----|
| B1 | D | C | A1 | B2 | A2 |
|----|---|---|----|----|----|

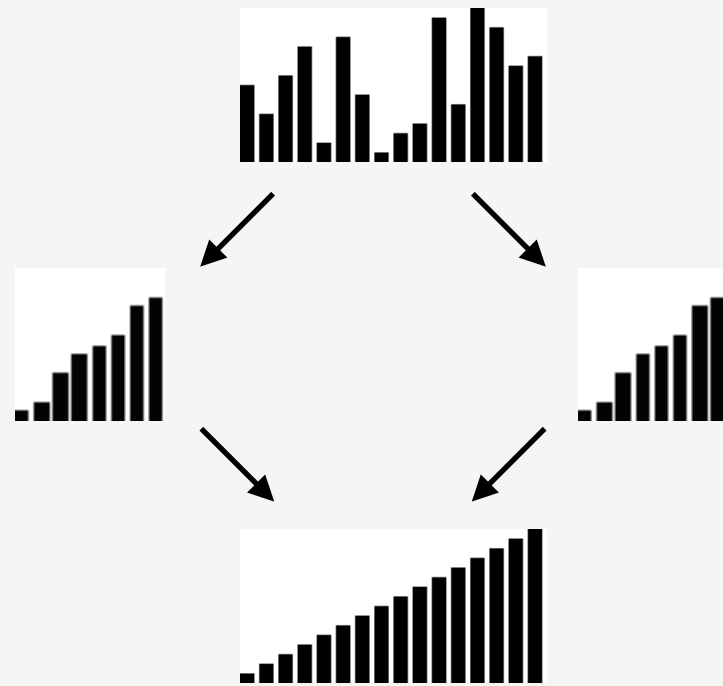
|    |   |   |    |    |    |
|----|---|---|----|----|----|
| B1 | D | C | A1 | B2 | A2 |
|----|---|---|----|----|----|

|    |    |   |    |    |   |
|----|----|---|----|----|---|
| B1 | A2 | C | A1 | B2 | D |
|----|----|---|----|----|---|

|    |    |    |    |   |   |
|----|----|----|----|---|---|
| B1 | A2 | B2 | A1 | C | D |
|----|----|----|----|---|---|

|    |    |    |    |   |   |
|----|----|----|----|---|---|
| A1 | A2 | B2 | B1 | C | D |
|----|----|----|----|---|---|

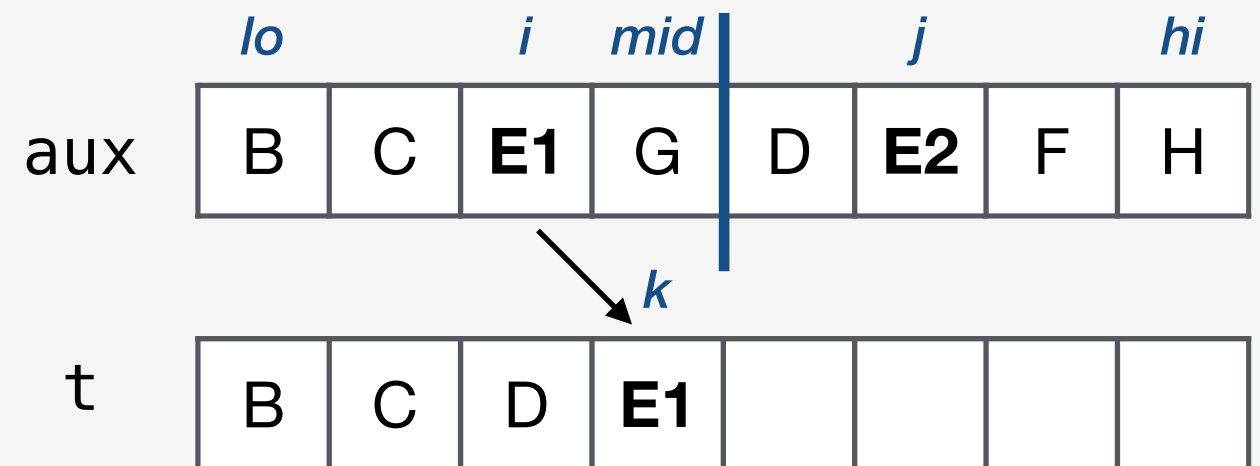
# Quels algorithmes sont stables ?



tri fusion

tri fusion est **stable**

```
int i = lo;
int j = mid+1;
for (k = lo; k <= hi; k++)
    if (i > mid)
        t[k] = aux[j++];
    else if (j > hi)
        t[k] = aux[i++];
    else if (less(aux[j], aux[i]))
        t[k] = aux[j++];
    else
        t[k] = aux[i++];
```



car le fusionnement est stable :

deux éléments ayant la même clé **E** restent dans le même ordre

# Comparaison

|                   | meilleur cas     | pire cas         | en place ? | stable ? |                                                               |
|-------------------|------------------|------------------|------------|----------|---------------------------------------------------------------|
| tri par sélection | $O(n^2)$         | $O(n^2)$         | ✓          | ✗        | n échanges                                                    |
| tri par insertion | $O(n)$           | $O(n^2)$         | ✓          | ✓        | petites données et clés partiellement triées                  |
| quicksort         | $O(n \log_2(n))$ | $O(n^2)$         | ✓          | ✗        | $O(n \log_2(n))$ avec pivot aléatoire, souvent le plus rapide |
| tri fusion        | $O(n \log_2(n))$ | $O(n \log_2(n))$ | ✗          | ✓        | $O(n \log_2(n))$ , peu de variation de la complexité          |



$O(n)$  après amélioration

# Comparaison

|                      | meilleur cas     | pire cas         | taille de tableau aléatoire<br>trié en 1 seconde / 20 secondes |             |
|----------------------|------------------|------------------|----------------------------------------------------------------|-------------|
| tri par<br>sélection | $O(n^2)$         | $O(n^2)$         | 55 000                                                         | 250 000     |
| tri par<br>insertion | $O(n)$           | $O(n^2)$         | 90 000                                                         | 390 000     |
| quicksort            | $O(n \log_2(n))$ | $O(n^2)$         | 10 000 000                                                     | 170 000 000 |
| tri fusion           | $O(n \log_2(n))$ | $O(n \log_2(n))$ | 6 500 000                                                      | 110 000 000 |

# En pratique

```
void qsort (void* base, size_t num, size_t size,  
           int (*compar)(const void*, const void*));
```

C

l'algorithme n'est pas spécifié par le standard  
qsort — souvent mais pas nécessairement implanté par quicksort

C++

l'algorithme n'est pas spécifié par le standard  
**introsort** (quicksort + **tri par tas**)  
tri par insertion pour des petits tableaux

Python

**timsort** (tri fusion + tri par insertion ++)



Java

**timsort** pour les objets (l'API spécifie : *"the algorithm used by sort does not have to be a mergesort, but it does have to be stable"*)  
quicksort (partitionnement amélioré) pour les types primitifs  
(la stabilité ne concerne pas les types primitifs)